The Hypothesis Graph: Semantic Memory Written by Methodeutics

Agent-native epistemics: verifiable knowledge linked to the work.

A Preprint

June Kim
kimjune01@gmail.com
ORCID 0009-0005-3153-9396

May 28, 2026

Abstract

A coding agent can return a fix and, with it, a typed record of how it got there that a stranger can replay. The instrument is the hypothesis graph: a persistent semantic memory whose nodes are falsifiable hypotheses bound to world-facing trials, whose edges are kill conditions that fire on evidence. It is written by methodeutics, Peirce's abduction, deduction, and induction run as a mechanical pipeline behind a deterministic gate no model arbitrates. Holding all three modes as kill-graded nodes is what the graph is for; the facet we dissect is the externally graded abductive step.

Agents already verify their work with two of those three modes: deduction in the compiler and the type-checker, induction in tests and benchmarks. The third is abduction, forming a hypothesis and hunting the case it cannot explain. It has been built into agents before (bi-abduction in Infer, the observe-hypothesize-test loop in Voyager), so the mode is not the novelty; what this harness adds is to run it as a first-class step graded from outside the model. That external grade is a math operation, the bi-abductive split of figure from ground computed as a symmetric difference against a ground-truth oracle the model cannot supply from its own belief (§4).

We dissect one inquiry where this decides the outcome. On a post-cutoff Verus compiler bug, kills graded by an external oracle drive the model off the narrow plateau six self-graded methods stop at, onto a more general fix bounded by what the gate covers; supply the coverage the gate missed and the model reaches the merged human fix's behavior on every graded probe, so the remaining gap is coverage and not capability.

The boundary the case exposes: a model can author the enumeration of cases but not the oracle that labels them, since that truth enters only from a world-facing trial, never from recombining what the weights already hold. Externalizing that one step delivers reasoning the model does not reach alone, and turns a machine's answer into an auditable trail a hostile reader reruns rather than a verdict they must trust, so merit attaches to the work and not the ephemeral agent.

1 Introduction

> Nullius in verba. Take nobody's word for it.
> — Motto of the Royal Society (1660)

The binding cost of a coding agent is increasingly verification, not generation. An agent returns a patch that passes the visible tests, but passing tests certify only the absence of visible failure, not correctness: an over-narrow patch passes them and is wrong off-suite. Establishing that the patch is correct requires reconstructing the reasoning the agent did not record, at a cost approaching that of producing it. The

reviewer is left to either accept on trust and absorb the latent defect, or re-derive the analysis and forfeit the saving the automation promised. The work is relocated rather than removed, from writing to checking. Checking another agent's undocumented reasoning is the harder task. The applied-AI literature is converging here: source-authority is failing as a trust signal, and a recent DeepMind position paper calls for "robust falsifiability pipelines" by name (Marchal et al. 2026).

A second problem compounds the first. The reasoning is not retained: each run rebuilds context from scratch. Even where agent memory adopts the cognitive-architecture lineage, as CoALA (Sumers et al. 2024) does in mapping it onto Soar (Laird 1987) and ACT-R, the semantic slot stores facts rather than a falsifiable structure, so the search is discarded once a patch passes and no trail survives to check.

We were promised a junior developer with near-infinite patience. A bare model on a minimal prompt is something else: an amnesiac, cracked-up contractor, sharp in the moment, with no memory of why it acted and no trail a reviewer can check.

Here we give it the trail. The hypothesis graph is a semantic memory that holds an agent's reasoning while it works, so the fix arrives with the inquiry that produced it: what was hypothesized, what was tested, what was ruled out, each step bound to a trial a stranger can rerun. The promise reverses the default. A bare agent hands you a verdict to take on trust; the graph hands you a chain to check, one that outlives the context window instead of dying with it. The model still does the reasoning, as it always did. What moves out of the weights is the method that holds that reasoning, types it, and keeps it replayable.

Here is the answer to each problem, backed by a receipt rather than asserted:

Problem	Solution
Reasoning cost	A domain-aware division of intellect: the model reasons, the harness holds the open hypotheses and runs the gate, the way you would not ask an LLM to keep a database in its context.
Agent trust	An attested chain with the discipline of accounting: a claim clears only when an independent party reruns its recorded trial, never on the agent's word, and nothing self-grades.
Lost reasoning	The inquiry is held in a legible representation, a kind of externalized interpretability, typed nodes a later run or a human reads and replays, instead of a transcript that evaporates once the patch passes.
Review bottleneck	Every step is grounded in a trial, so a reviewer accepts the work or resumes where it stuck, instead of judging it ungrounded, tossing it, and reassigning to a human.

What memory does a coding agent actually need? Not the kind the word usually means. Agent memory has mostly meant keeping context: transcripts, retrieved chunks, the last n turns. But coding is, almost entirely, reasoning. The binding work is diagnosing why something fails and deciding what will fix it, not typing the patch. And for a reasoning task the only context worth keeping is the reasoning trace itself; the transcript and the files read are recoverable or beside the point.

So the memory a coding agent needs is not a store of what it saw but a record of what it worked out, and the existing structures each keep something else. Prose context is lossy, skill libraries store verified code rather than falsifiable claims, vector retrieval indexes established chunks, and cognitive architecture, which defined the typed memory slots, kept filling the semantic one with facts. The missing piece is a place to hold a hypothesis under consideration, and the gap is hypothesis-shaped.

Made precise, the graph is a typed record of an inquiry in progress: candidate causes as nodes, falsification conditions as edges, every belief carrying the mode that earned it and the trial that can re-earn it, a data structure at the harness layer, just a markdown file with no RAG, no database, zero dependencies (§2).

It fills the semantic-memory slot (`smem`), with the methodeutic skills as procedural memory and per-run trajectories as episodic, and the model plugged in as the inference component, a reasoner inside a method it does not own.

This reframes the usual answer to how do you make a model reason better. The industry's reply is almost always more: more parameters, more training, the inference dial from high to ultrathink, reasoning sold by

the token. The hypothesis graph is the other answer. It puts the method of inquiry outside the weights. The part that does the most work is the one a model cannot buy with more compute: the kill condition's external ground-truth oracle, the harness's way of externalizing abduction (§4). And the machine produces, with the fix, an attested chain of mechanical reasoning to reach it, each step a node bound to a recorded trial. One post-cutoff, maintainer-stuck compiler bug carries both claims, and the chain it leaves behind is auditable end to end, a property no benchmark verdict supplies (§7).

So here are the four claims:

1. A port: the counterexample-guided refinement loop of formal methods (CEGAR/CEGIS; model-based diagnosis), carried from a closed abstraction domain into the open, LLM-populated semantic-memory slot of a cognitive architecture for coding agents, with replay as the per-node soundness the lost abstraction no longer supplies (§2, §4, §16).
2. A mechanism, dissected: abduction externalized: the hypothesis graph's power is in its kill edges, and the kill condition's source decides how far a climb generalizes. On a post-cutoff Verus compiler bug, a graph whose kills are graded by an external oracle drives the model off the narrow plateau and onto a more general fix bounded by the gate's coverage, where six methods whose kills are graded against the model's own belief plateau on the narrow case; supplying the missing coverage then carries the model to the merged human fix's behavior on the graded probes. The dissection locates the encoding boundary: the model can author the enumeration a kill ranges over but not the oracle that labels it, so kills graded from outside deliver the abductive step the model does not reach alone. This is one fully-instrumented, replayable inquiry, an existence claim for a mechanism that can occur, not a rate or a general boundary claim (§7, §11).
3. An epistemic reframe: agents should not be trusted, they should be accountable, and the hygraph is the ledger that makes them so (§9).
4. A null result, correctly typed: run on the field's standard benchmark, the assembly resolves 95.3% of the public split, and that number answers nothing claimed here. The obvious question, does the method improve benchmark performance, is not applicable: the bench grades spec-conformance against a visible oracle, so the typed diagnostic reasoning the hygraph encodes is precisely what it cannot see. Every attribution channel comes back near zero, and the null is the instrument's blindness, not the method's failure (§8.2, §8.4, §8.7).

The instrument is the systems contribution; the epistemology is what it makes possible: a position on what makes a claim checkable, what agents owe the people who act on their outputs, and where merit attaches. The instrument is concrete, the epistemological claim primary, and it survives exactly where the benchmark result nulls.

This work is independently funded by the author. Its only standing is the auditable trail: every claim above ties to a committed receipt, the nulls included.

2 The hypothesis graph

The constraints come straight from the two problems the introduction posed, a confident output no one can verify more cheaply than by reproducing it, and reasoning that does not survive the run. A semantic memory that answers them must clear four requirements at once, and no prior structure clears all four:

- Holds hypotheses. Stores a hypothesis still under consideration, where prose, skill libraries, and retrieval keep only verified facts and established chunks, the hypothesis-shaped gap the introduction named.
- Kill condition. Each claim carries the executable test that can prove it wrong, the falsifiability the introduction's reviewer had no way to run.
- Independently verifiable. A stranger verifies a conclusion by rerunning its recorded trial, instead of re-deriving the reasoning or taking the author's word, the dilemma the introduction left the reviewer in.
- Persistent memory. The trail persists past the context window rather than being discarded once a patch passes, the retention the introduction's second problem named.

Prior structures clear part of this; the table scores the strong version of each against the four (Y native, ~ with common extensions, N needs another layer doing the work).

Structure	holds hypotheses	kill condition	independently verifiable	persistent memory
Hypothesis graph (this work)	Y	Y	Y	Y
Truth-maintenance (Doyle 1979; de Kleer 1986)	Y	~	~	~
Provenance / lineage (W3C PROV, Moreau et al. 2013)	~	~	~	Y
Search + proof tree (Clarke et al. 2000; Solar-Lezama et al. 2006)	Y	~	~	~
Argumentation (Dung 1995; Modgil & Prakken 2014)	Y	~	~	~
Event-sourced log / ReAct trace (Yao et al. 2023)	Y	~	~	Y

The columns are the introduction's two problems turned into tests: the middle two are verification (a falsifiable trial a stranger can rerun), the outer two are the hypothesis-shaped gap and retention. Each structure clears part; only the hypothesis graph clears all four in one append-only file.

What can finally satisfy all four together? The LLM. Filling the graph takes a reasoner that reads a surprising failure, proposes candidate causes in open vocabulary, and turns each into an executable test, with no hand-built domain model. Classical inference engines could do this only inside a formalism encoded by hand, which is why the slot stayed a research program; the LLM populates it across arbitrary codebases, which is what makes the structure practical here rather than a paper exercise.

The structure meets all four with three pieces, a node, an edge, and one invariant. A node is a claim bound to a trial: a hypothesis, the perturbation that tests it stated as an exact command, the observed outcome, and a credence typed by the reasoning mode that established it. An edge is generated by a kill condition: the manner of a hypothesis's death names the next hypothesis, so the structure is self-extending, with no external controller deciding where to look. The invariant comes before the operations, because every operation is defined to preserve it. State it as a standalone contract:

> Replay invariant. Every node is reconstructible, by a stranger who does not trust the author, from its recorded trial alone: the exact command, the observed outcome, the verdict, and the credence cap.

The contract holds on the node's mechanical skeleton; the hypothesis prose the node also carries falls outside it. That split is deliberate. The prose is the part a reader would otherwise have to trust; the recorded trial is what replaces trusting it, so the invariant draws the line exactly where checkability begins and leaves the unformalizable payload outside it. The guarantee is narrow and named: the command, outcome, and verdict are checkable, while the mode label that caps a credence is a convention the writer is trusted to apply honestly.

Five operations maintain the structure, each defined with the one-clause argument that it preserves the invariant, the way a balanced tree's insert is defined to restore balance:

- Create (a smart constructor), append a node: abduction writes a hypothesis with its kill condition and the exact trial that tests it ("the bulb is dead", trial `swap in a fresh bulb`). Preservation: a node is admitted only if its trial replays, so by construction every committed node satisfies the contract, and the invariant holds by induction over reachable graphs. This admission rule is what makes every later operation safe.
- Read / replay: ask which hypotheses are still open, and reconstruct any node by re-running its recorded trial, no trust in the author required. Preservation is vacuous, read mutates nothing, but replay is the load-bearing operation, the check every other operation's preservation is stated against.

- Classify (the update): a trial's outcome marks its node killed or witnessed and caps its credence at the mode that earned it, a verdict written once. Preservation: classify appends a verdict and never edits the recorded trial, so the node still replays to the same outcome.
- Link (edge-from-kill): the manner of a hypothesis's death names the next hypothesis (the dead bulb names "the dimmer is broken", trial `bypass the dimmer to the wall`), so a classification spawns the next node with no external controller. Preservation: link only appends, and each new node is admitted under Create's rule. This is the one genuinely novel operation, and the search path it builds is the justification.
- Prune: a dead branch leaves the working frontier while its record stays in place. Preservation is trivial, prune is a frontier-set operation that changes what is live and deletes nothing, so every pruned node replays exactly as before.

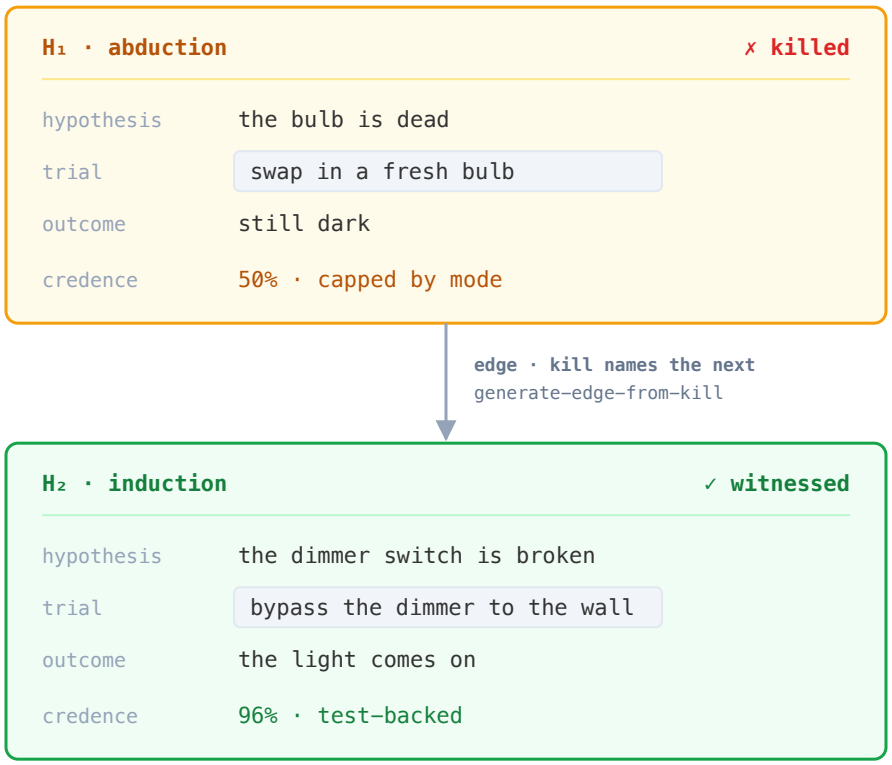

Figure 1: A hypothesis graph, two nodes and the edge between them, on the dead-light inquiry. The bulb hypothesis is killed by a cheap trial (swap in a fresh bulb, still dark); its death names the next node, the dimmer, which a second trial witnesses (bypass it to the wall, the light comes on). Each node binds a hypothesis to a trial, an observed outcome, and a credence capped by the mode that earned it: abduction proposes and stays low, induction is test-backed and rises. Every node rebuilds from its recorded trial, so a reader replays the structure instead of trusting it.

The payoff is the central claim of value, and it follows from the invariant rather than from any benchmark:

> Local Replay Auditability. Any single conclusion is checkable by re-running that node's one recorded trial, without reconstructing the inquiry and without extending trust to whoever ran it.

This is the analogue for inquiry of a certificate a consumer checks without trusting the producer, and like the Merkle audit path or proof-carrying code, its value rests on a contract; a complexity bound is beside the point.

Two grades of it matter. Where the trial is a deterministic command over pinned inputs, a compiler on a fixed toolchain, a test in a container, replay is strong: re-execution reproduces the recorded outcome, and the lead case (§7) is here. Where the trial runs a model or a live service, replay is artifact-level: the recorded output is verified and the deterministic predicate re-run over it, the command standing as a provenance event rather than a reproducible computation. The honest scope is the first shading into the second as the system under test grows less deterministic. Pruning leaves all of it untouched: a pruned branch drops from the working frontier but stays in the record, so its nodes replay exactly as before.

The nodes are ordinary; what is novel is the edge semantics. A search tree finds; a proof tree justifies. The hygraph is both at once, because the search path is the justification: every step was a trial.

It sits at the confluence of older lineages: truth-maintenance and model-based diagnosis (de Kleer 1986; Reiter 1987; de Kleer & Williams 1987), sequential experimental design (Wald 1947; Vovk & Wang 2021), abstract argumentation (Dung 1995), and counterexample-guided refinement (CEGAR, Clarke et al. 2000; CEGIS, Solar-Lezama et al. 2006). Refinement is the closest kin: a counterexample is a kill that names the next experiment, the hygraph's defining edge. What is novel is not that loop but running it over an open hypothesis space abduced in domain vocabulary. Replayability stands in for the sound abstraction a closed setting supplies for free, with completeness as the price the open move forfeits (§16).

The near neighbors each hold part of this and source the rest from outside themselves. A truth-maintenance system (Doyle 1979; de Kleer 1986) maintains belief status under assumptions, but the empirical trial and the kill-generated successor are external conventions. Provenance (W3C PROV, Moreau et al. 2013) records replayable activities, yet does not decide which hypothesis comes next. A search-plus-proof setup (Clarke et al. 2000; Solar-Lezama et al. 2006) can make the path both find and certify, but leans on a domain proof calculus and an external search policy; argumentation frameworks (Dung 1995; Modgil & Prakken 2014) structure attack and defeat but bind to no executable test; and the ReAct trace (Yao et al. 2023), the strongest mundane baseline, is an append-only log whose continuation policy the controller decides and the record never holds.

What the hygraph adds is their composition in one append-only object: an open-domain hypothesis, its executable trial, its kill condition, and the successor that kill names. That object belongs to the verifiable family whose value is a contract rather than a complexity bound, certificate transparency (Laurie et al., RFC 9162), proof-carrying code (Necula 1997), content-addressed provenance: a data structure paired with the protocol that writes and checks it.

This is the data structure for testable inquiry, and its entire power is the perturbation surface. Strip the ability to poke the system and read an outcome, and the same shape degrades into a plausibility tree, which is the confabulation failure mode it exists to prevent. It is also not how minds run. Minds run on simulation, fast and compressive and intuitive, but a simulation is not verifiable from outside, so inquiry that has to be checked trades it for an explicit perturbation surface. The hygraph is the verifiable serialization reasoning compiles to, so it can be checked by someone who does not trust you. Proof is to intuition as the hygraph is to inquiry: not the thinking, the residue of the thinking that survives a stranger's replay.

That residue is what the memory typology calls the `smem`: persistent, typed, queryable, and owned by the harness rather than the model. Those same properties make it an interface for agent interop: because the structure is typed and external, a second agent, a later run, or a human auditor reads and writes against one contract and can rerun any node rather than trust it (§9). The graph in this work is one markdown file per inquiry. It was never the bottleneck at any repo size in the eligible set; no vector store or graph database was needed (§11).

3 Theoretical grounding: methodeutics and a runnable epistemology

How can reasoning be mechanical? The insight itself, the leap to a candidate cause, stays with the model. What the harness makes mechanical is the method. Reasoning becomes mechanical the way proof is: not the having of an idea, but the discipline of checking it. In a precise and limited sense the harness encodes a mechanical discipline of inquiry, not a theory of mind.

The procedure is encoded method, living in the harness rather than the weights, and it separates the modes that earn a belief. Peirce named them before any of us were born.

His Illustrations of the Logic of Science (1878) and Pragmatism as the Logic of Abduction (1903) type the operations of inquiry into three modes that are not interchangeable.

- Abduction generates explanatory hypotheses for surprising observations: what would, if true, make this no longer surprising?
- Deduction derives testable predictions from hypotheses: if this hypothesis holds, what follows?
- Induction tests predictions against evidence: does the evidence accord with the prediction?

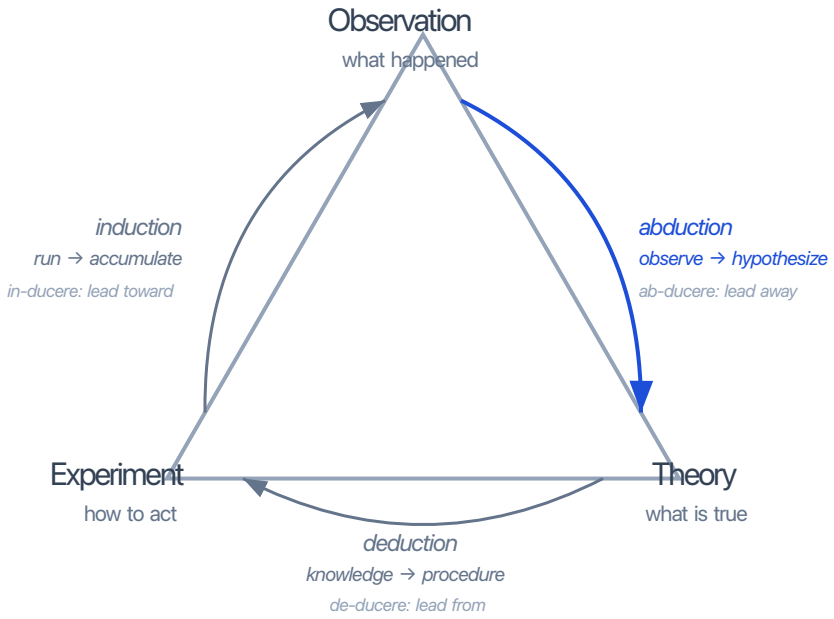

Figure 2: The three modes as one cycle: Observation -> Theory (abduction), Theory -> Experiment (deduction), Experiment -> Observation (induction). `inquire` traverses all three before any code is written; a partial traversal is a partial inquiry.

No single mode carries a belief to its grade. Abduction proposes content but does not test it; induction tests but introduces no new explanatory content; deduction traces consequences but invents nothing. The credence a node ends up with is what traversing all three earns it, and that is what it means to call the modes typed: each is fixed by what it can't do. Keep them separate and each does its one job; collapse them and you get familiar failure modes:

- Confirmation bias: induction without abductive alternatives
- Confabulation: abduction without inductive grounding
- Free-association: no typed mode at all

That collapse is exactly what modern LLM agents do by default, since a single forward pass proposes, predicts, evaluates, and rationalizes in undifferentiated prose. Methodeutics, Peirce's term for the methodology of inquiry, is how to conduct the typed-mode loop well. Encoded as skills, it is the `pmem` of this architecture: the procedural memory whose job is to construct and maintain the `smem`.

Modes of reason and the irreducible three. Around the act of testing, philosophy of science built an apparatus of real rigor: Bacon's induction (1620), Popper's falsifiability (1934), Meehl's "soft science" critique (1967), Pearl's causal calculus (2009). Justification got its method, every step of it. But it begins one step too late, taking the hypothesis as given and filing its origin under inspiration. The discipline built an epistemology of justification and none of discovery. Peirce alone named the operation, abduction, and was ignored. The harness runs it as a first-class typed mode.

An epistemology a machine can run. Peirce's modes type how an inquiry moves; they do not say what a node believes or what would make that belief true. That node semantics is its own question, and the full answer, why justified true belief does not run as a procedure and what buildable parts replace it for a machine rather than a human knower, is developed separately in Verifiable Knowledge, the agent-native epistemics this node semantics presupposes, and its frame What Cannot Be False Cannot Be True. Several groups are now reaching the same territory (§9 returns to DeepMind's artificial epistemic agents, and §10.3

to Traxia). The harness needs only that epistemology's operational core, small enough to state in a line: warrant is truth. A node is true when its build presently passes a trial a stranger can replay, false when the trial has fired, and untrue when no trial has yet landed. The rest of this section is that identity made local to a node and its kill edge.

Belief and knowledge, in brief. The full arc is in the standalone paper's belief and knowledge sections; the harness uses two facts from it. First, a node carries a credence rather than a boolean, capped by the mode that earned it: abduction held low because it only proposes, induction allowed higher because it has been tested (Ramsey 1926). This is the rung a bare LLM fails by default, emitting uniform apparent confidence with no propagation along the chain, the confident confabulation the typing and the kill conditions exist to prevent. Second, knowledge is not a tier above belief but a node that has survived its trial, belief that ran a real chance of falling and is still standing, indexed to the stakes of acting on it.

Truth, made of parts. Warrant is truth, and its parts map onto the hygraph one to one: provenance is the dependency graph, a citation is an edge, attestation is the recorded trial, falsifiability is the capacity to go red and is the kill condition, the test is the world-facing trial, and reproducibility is replay (Truth Is Buildable).

Truth does not live in the nodes. A node wired to nothing is a tautology, irrefutable and useless at once, the irrefutability and the uselessness one property. This is the sibling paper's title made mechanical, what cannot be false cannot be true: the capacity to fail comes first, and a node earns the right to be called true only by surviving a kill it could have fired. Truth lives in the edges, the kill-conditioned ones that carry the warrant.

The guard against relativism is that same edge: a build that includes a test it can fail is the opposite of a hardcoded return value, which is the rigged benchmark wearing a decimal point.

That is where the node states get their meaning. A witnessed node is a build presently passing, true. A killed node is a build that went red, false. An open node is a conjecture awaiting its test, neither true nor false but untrue, the honest status of every hypothesis before its trial lands. The three states, true and false and untrue, are a ledger of warrant rather than a new logic (the entitlement ledger and its three states are developed at length in the companion paper), and the distinction is deliberate: the contribution is not a third truth value but the discipline of recording which of the three a claim has earned and binding that record to a trial a stranger can rerun. The replay invariant of §2 is this epistemology made local, the reproducibility rung of a buildable truth scoped down to one node and its kill edge.

Peirce's modes are the operations, the entitlement ledger is what a node means, and both are runnable because every rung is a buildable, checkable structure. That mechanical discipline of inquiry, the unit and its derivation, is the claim we test here, and the ground we test it on.

4 Methodeutics, applied

Abduction completes the trichotomy as an idea. To put it to work, you need a way to generate a hypothesis and a place to hold it: a diff, and a typed graph.

Bi-abduction, tri-abduction, and compositional inference. The primitive under abduction is a diff: a before snapshot, an after snapshot, the flip as figure and what held as ground, Rubin's Gestalt terms. Arity grows from there, from unary (one before/after pair, one frame, the ground held fixed) to bi-abduction (the frame inferred autonomously: Calcagno et al. 2009, O'Hearn 2019, scaled industrially in Facebook Infer) to tri-abduction (a diff across branches: Zilberstein et al. 2024, Outcome Separation Logic). `inquire` worked at the unary-to-bi level, a single before/after diff with the frame inferred from the symptom.

The "XOR" used here throughout is shorthand for exactly that bi-abductive separation, the figure (what the fix must change) held apart from the ground (the frame that must stay invariant). Where bi-abduction infers the frame to make a proof go through, the harness fires the same split as a check: it computes the figure-from-ground difference against a ground-truth oracle and keeps only the cases the difference flags. The symmetric-difference framing is the operational form; the inference it operationalizes is bi-abduction (Calcagno et al. 2009; O'Hearn 2019).

Directed graphs as reasoning representation. Pearl 1988 (Probabilistic Reasoning in Intelligent Systems; Bayesian networks as DAGs of dependencies); Pearl 2000/2009 (Causality; structural causal models, d-separation, do-calculus). Our data structure (typed nodes, directed edges) applies Pearl's lineage to hypothesis representation rather than causal-structure inference. The difference from a Bayesian network is one of kind, and runs deeper than dropped probabilities. A Bayes net conditions over a fixed variable set and

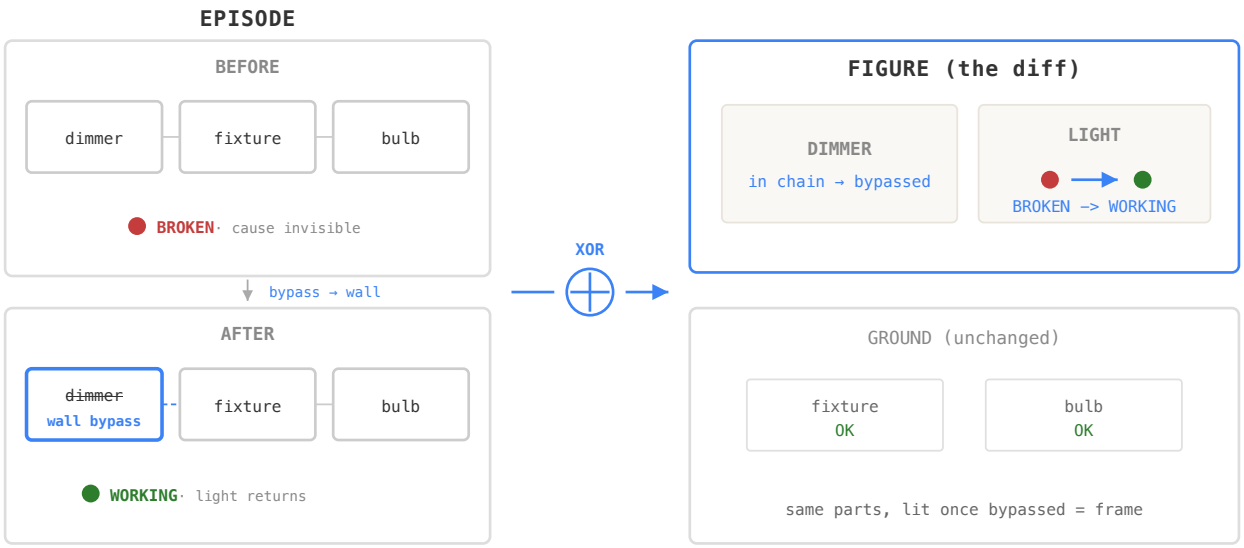

Figure 3: An example of bi-abduction. The symptom is a light fixture unresponsive to switch input, and it underdetermines its cause: dimmer, fixture, and bulb are all intact, so the static scene names no suspect. The perturbation manufactures the second snapshot, bypassing the dimmer to the wall, and the XOR isolates the figure (the dimmer) from the ground (fixture and bulb). Generating that diff is the abductive act; induction follows, convicting the dimmer once the light returns.

propagates probability along edges of dependence; the hygraph abduces its nodes as the inquiry runs, and its edges are genealogical, a dead hypothesis naming its successor rather than a conditional dependence. A Bayes net is justification over a space it is handed; the hygraph generates the space.

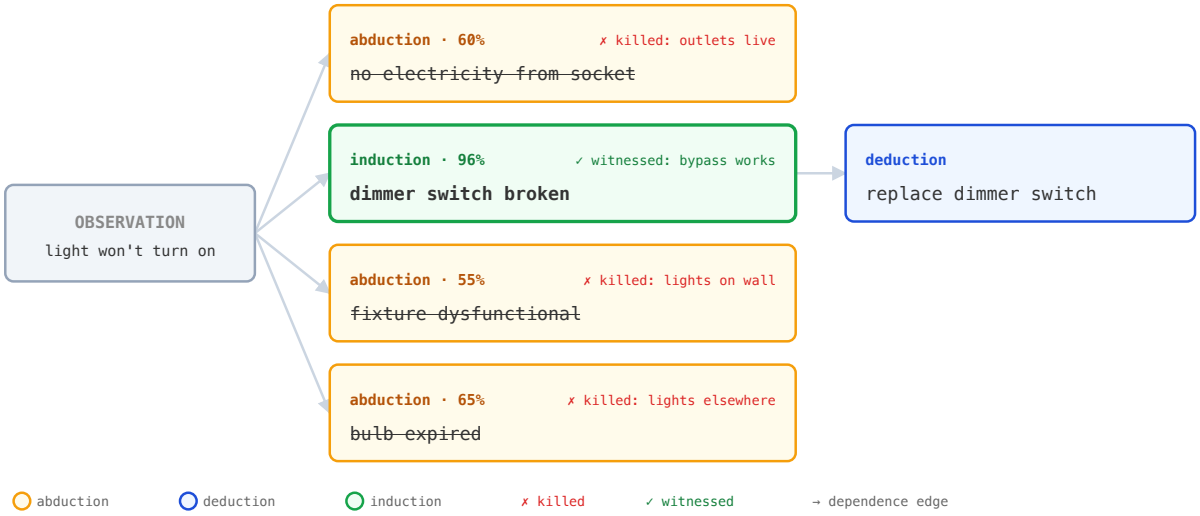

Figure 4: The hypothesis graph for the dead fixture. Abduction fans the observation into four typed candidate nodes; mechanical kill predicates fire on three (the socket, fixture, and bulb each cleared by a cheap test), the dimmer node is witnessed by the bypass and closes the last open hypothesis, and deduction derives the fix. Typed nodes, directed edges, all three modes in one inquiry.

Isn't generating that space just debugging? It is, and debugging has been automated for decades: spectrum-based fault localization, statistical and delta debugging, model-based diagnosis, and search-based program repair are mature fields (Jones et al. 2002; Liblit et al. 2005; Zeller & Hildebrandt 2002; Reiter 1987; Le Goues et al. 2012; Monperrus 2018). Debugging tools already automate the loop; what is new is that the

hygraph persists it. Every engineer, and every repair tool, runs some version of abduce a cause, kill it on evidence, witness the survivor, derive the fix, but the engineer runs it in their head and the tools, whatever logs they keep, do not persist the search as a typed, replayable hypothesis graph, the trials collapsing into a verdict once a patch passes. The harness gives the loop a typed, replayable substrate, the hygraph, and a deterministic engine to run it, so a model executes the inquiry and a stranger can replay every step of it.

Composition over the hypothesis graph. Three lineages enter through separate roles. The structural skeleton is Pearl's DAG: typed nodes, directed edges, the probabilistic semantics left behind. The node semantics is credence (Ramsey 1926; James 1907; Dewey 1929): each node carries a belief at a credence level, capped by its reasoning mode, continuous rather than boolean. Confident confabulation, high confidence on a node that hasn't earned it, is the failure mode the stage-typing and kill conditions jointly prevent. The update semantics is the binary test verdict, written back to the active hypotheses as a kill or witness, together with a re-entry route the driver follows (§5.4). Pearl's skeleton, Ramsey's credence, a deterministic verdict->route update: three primitives, three roles, one data structure.

5 The harness

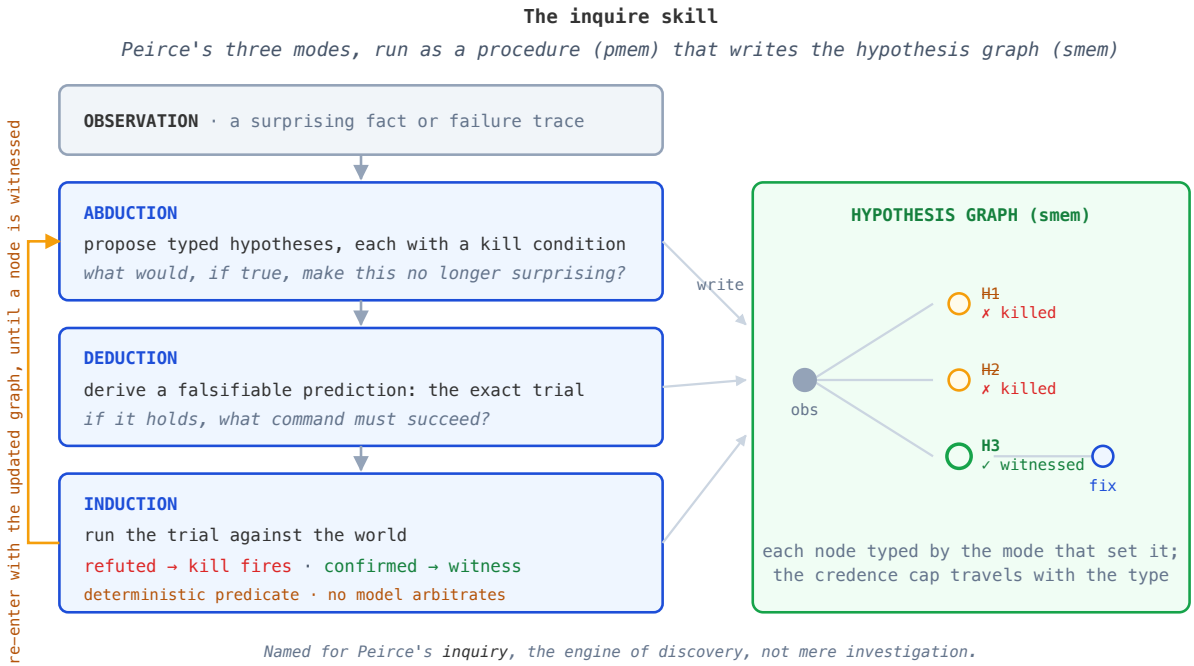

Figure 5: The `inquire` skill: Peirce's three modes as a procedure (`pmem`) that writes the hypothesis graph (`smem`). Abduction proposes hypotheses with kill conditions, deduction derives the trial each must survive, induction runs it and fires a deterministic kill or witness with no model arbitrating. Each node is typed by the mode that set it, the credence cap travels with the type, and the loop re-enters with the updated graph until a node is witnessed. `implement` and `attest`, which read the survivors and verify the patch, follow below.

Throughout, the three stages are `inquire`, `implement`, and `attest`; the frozen artifact's code, file paths, and route literals spell them `recon`, `craft`, and `audit`.

5.1 The inquiry frame

We recast each issue as an inquiry on an engineered system: a failure trace, a codebase, a root cause to find, and an intervention that must not regress the rest of the system. Code is the right substrate for the hygraph because it combines three properties that other inquiry domains rarely combine:

- Reproducible: same input yields same output, modulo controlled nondeterminism
- Deterministic: causal lines from input to behavior are mechanical
- Perturbable: single-line and single-function diffs are cheap to apply and fully observable

Because those three hold together, hypotheses about code can be tested by cheap mechanical perturbations and falsified by deterministic predicates; kill conditions are not approximations; they are executions.

One trial settles the predicate in this regime. In code the per-case response is mechanically observable, so a single passing test on a captured diff is a complete verdict that the diff satisfies the executable predicate for that case. That verdict speaks to the predicate alone: behaviors it doesn't cover are out of scope, a boundary that §7 shows is exactly where the differences live. Where such verdicts are aggregated, the right summary is counts and denominators rather than confidence intervals: per-case verdicts are exact, and aggregating them is bookkeeping.

The three Peircean modes are how `inquire` builds the graph, each node typed by the mode that established it and capped at that mode's confidence:

Mode	What `inquire` does	Confidence
Abduction	Proposes candidate root causes from the observed failure; writes hypothesis nodes with falsifiable predicates and kill conditions (read-only)	low
Deduction	Traces each hypothesis's consequences through the code to localize the suspect set	high
Induction	Tests survivors with cheap read-only experiments (prints, intermediate data)	moderate

`implement` then writes the surviving hypothesis, with an adversarial challenger critiquing the diff against the spec. `attest` runs the test suite, takes the grader's pass/fail verdict, and emits a re-entry route (`inquire`, `implement`, or `none`) from a fixed verdict->route table. The driver parses the verdict and the route; both are mechanical, and no model decides termination.

5.2 Hypothesis graph as inquiry output

`inquire` emits the hygraph: the structured-analysis document that precedes the patch. Kill conditions are mechanical predicates over the evidence trajectory, not model preferences, so a node dies when its predicate fires and not before. The graph persists across iterations; re-entry adds nodes rather than overwriting. The frontier closes only when every open hypothesis is killed (a test refutes it) or witnessed (a test confirms it).

A committed node is a conclusion, and an inquiry that reaches one rarely runs straight. Following the `inquire` skill on a real bug, a single hypothesis flips across all three modes and a kill before it settles:

> abduction -> deduction -> kill -> abduction -> deduction -> induction -> deduction -> induction => induction , 93%

An in-flight inquiry trace, illustrative: Sonnet 4.5 following the ***`inquire`*** skill on the python-dotenv ***`find_dotenv`*** v1.0.1 regression (a real, reproducible bug, every command run). The active hypothesis cycles through all three modes and a kill before the inquiry settles; a committed graph records only the terminal node (induction , 93%) and discards this sequence. Full trace: recon-inflight-dotenv.md. Not a frozen Pro instance.

5.3 Blind cross-model challenge at the hypothesis stage

To keep that hypothesis from being one model's artifact, two frontier models from different families receive the same evidence pack with no cross-visibility, and each produces a hypothesis independently. A third pass extracts the disagreements: the disagreement becomes the next node in the graph, while the agreement is recorded but not actionable. Adversarial filtering operates at hypothesis time, while the worktree is still untouched, rather than at patch time where the diff is already written. Sampling stochasticity alone produces real divergence even within a single model; cross-family divergence compounds it with architectural and training-corpus differences. Both are signal.

5.4 Deterministic gating and the outer loop

The control loop is standard. `attest` prints a verdict and a re-entry route from a fixed table; the driver routes on those two lines with no model in the decision and under a bounded attempt budget; a failure re-

enters `inquire` with the updated graph rather than retrying the patch, so the hygraph doubles as the loop's checkpoint and no dead branch is re-proposed. The gate's reliance on a cheap test oracle is not incidental: §8.4 shows it does most of the work the bench number seems to credit to diagnosis.

6 Procedure

Where do you study the mechanism? Where a benchmark cannot reach: an undiagnosed bug whose correct fix is a predicate, not a case, with no cheap oracle and a genuinely hidden cause. That regime was not drawn to flatter the method. The determinacy audit (§8.7) marked it out independently, and before these results, by finding that the standard bench hands over the spec and a cheap visible oracle, which leaves a diagnostic method nothing to separate on; the regime here is the complement the audit defined, not a place chosen because the method wins there.

Far from a corner case, that regime is most of software, every issue tracker a backlog of undiagnosed problems waiting on the one expensive step a benchmark never exercises. The mechanism experiment (github.com/kimjune01/hygraph-mechanism) reconstructs that regime under control and runs the contrast a benchmark cannot, on a single bug dissected end to end rather than a population rate.

The contrast: self-graded methods versus an external oracle. The variable is where a kill's ground-truth oracle comes from (§7). On one side are methods whose kills are graded against the model's own belief: six prompt-encoded approaches, the hypothesis-graph inquiry among them, each run three times. On the other is a single method whose kills are graded against an external oracle, the `abductor` gate. The model, the loop, and the bug are held fixed across all of them, so any gap traces to the oracle's source, with the model's strength held constant.

The bench and its two goldens. Grading uses a tiny, hand-built bench for the one bug, scored against goldens from two deliberately separated sources, the free oracle of construction for the bug arm and the human-approved oracle for the hard arm; the mechanics are in §7.2. The model never sees a held-out probe's assertion, only a pass/fail handle, so nothing can pattern-match the test, and every verdict is from a forced-fresh, fingerprinted build.

A historical PR, taken post-cutoff. The case is a real merged PR from after the solve models' training cutoffs, so its fix sat outside the reachable corpus at solve time. The commit dates, the contamination-controlled model, and the recall probe that confirms reconstruction over recall are in §7.2.

Selection and preregistration. Candidates are drawn from the localization-hard band the lead case and its two neighbors define (§7.6): bugs where the symptom sits far from the cause and the project's own suite cannot tell a narrow fix from a general one. The protocol preregisters one sentence per loop, testing X, predict Y, refuted by Z, before any arm runs (`METHODOLOGY-preregistration.md`).

7 Results: the mechanism, in full

On a bug of that kind the agent's failure mode is specific: it patches the example in front of it, agrees when told to generalize and stays narrow, and writes the tests it already believes. An externalized check breaks that loop, and one bug shows how, dissected from the trace rather than summarized from a score. It makes the program's central claim concrete: reasoning can be encoded outside the model and delivered to it. The reasoning that generalizes the fix is not one the model's weights or prose reach here, both of which plateau; it is encoded in a general instrument, and applying that instrument carries the model to a rule it does not reach alone. A mechanism is shown by dissection, so the unit of evidence is one fully instrumented inquiry, traced line by line.

The contrast. Every arm here writes a hypothesis graph; what varies is where its kill edges get their truth. A kill's oracle (the ground-truth source that says which cases are correct, in the metrological sense of an instrument checked against a reference, distinct from the probability-calibration of confidence scores) is internal when the model grades the cases against its own belief, so the edge that retires a hypothesis fires on the model's say-so. It is external when the cases are graded against a known-good reference the model cannot see, so the edge fires on a disagreement outside its belief. That distinction is the whole axis. The carrier does not matter to it. Whether the external oracle arrives as a paragraph of prompt that hands over the answer key, or as a CLI tool that holds it, is incidental; what matters is only that it is encoded at the harness layer, outside the weights and outside the model's ephemeral reasoning. Six methods here grade against the model's own belief, the prose hypothesis-graph inquiry among them; one grades against an external oracle.

The data structure is the same across all of them, which is exactly why it cannot be the differentiator. The external oracle moves this number. The graph is the persistent, replayable substrate every arm writes into, the object contributed here and the reason any of this is auditable at all; the mechanism experiment isolates, within that substrate, the one variable that decides how far a fix travels. And externality is necessary, not sufficient. An external oracle with thin coverage drives a fix wide but leaves it broken (§7.4), so the axis is not external-versus-internal alone but whether the externally supplied labels span the distinction the fix has to make.

7.1 The instrument is general, and blind to the answer

The lift is reasoning-encoded-as-instrument only if the instrument did not smuggle in the fix, so this is established first. It did not. The instrument, released as `abductor`, has three domain-general operations and no answer built in. It enumerates a fixpoint-closed space of cases over the property's type-formers, wider than any one hypothesis. It calibrates each case against a known-good baseline, so the ground truth is external to the model. It gates on a single pass/fail signal the model hill-climbs against. The instrument never names the predicate, the property, or the fix; the prompt that drives it demands generality and supplies the signal, and nothing else, so reaching the rule is the model's own reconstruction.

Two controls establish the blindness rather than assert it. The model received a prompt naming no property and still had to grep out rustc's own machinery for uninhabited types to climb. And handed only the vague instruction to range over type-formers and judge, a second model rebuilt the instrument itself, a 7026-case enumeration, which is what a general construction permits and a bespoke oracle does not. The construction also transfers across domains: the same enumerate-calibrate-disagree shape, pointed at an unrelated Go SBOM tool (syft #4760), surfaced eleven omitted cases where manual review had found four, and exposed a tabulation bug inside the comparison criterion itself. Only the grammar of cases changes between domains; the machinery is fixed. What every instance exploits is one general object, a disagreement, the symmetric difference between what the system believes and what is true, the check that tests and type systems cannot supply because absence has no test.

7.2 The tiny bench, and what counts as golden

The case is one bug, so the bench is one bug, hand-built and small enough to state in full. It holds three states of the compiler, the buggy commit (base) and the maintainer's two later fixes (the narrow one and the general one), and a handful of probe programs, each of which the compiler either accepts (VERIFY) or rejects (REJECT). A probe's golden is the verdict a correct compiler owes it: REJECT for an unsound program, VERIFY for a sound one. Two sources supply those goldens, and keeping them apart is the method, because they are exactly the easy and hard arms of the result.

The case is a real historical PR, taken deliberately from after the cutoff. Verus #2219 is an issue the maintainers filed and closed, and its three states are real commits: the buggy base (2026-03-08), a narrow fix merged the next day, and the general fix merged on 2026-06-05. We chose it because those fixes postdate the solve models' training cutoffs (the contamination-controlled model is Fable, a January 2026 cutoff, against March and June 2026 fixes), so the merged solution was absent from the reachable corpus when the weights were frozen. A recall probe confirms it: asked in isolation how #2219 was fixed, the model does not recover the fix and says so. What it can reconstruct from general, pre-cutoff competence is that rustc carries uninhabitedness machinery at all, which is old and well-known; what it cannot recall is the merged fix for this bug. So the model's solution is reconstruction from prior competence, not recall of the post-cutoff patch. That is the property the case needs, and the commit dates and the probe transcript are committed alongside the dataset.

The bug probes are uninhabited programs by construction (an erased `!`, an empty enum, and two shapes the gate never sees: an associated-type projection and a nested generic). Their golden is REJECT, and the base compiler is wrong on exactly them, verifying what it should reject. That is the bug, and it makes base a free reference for the easy arm: the externalized gate enumerates such cases over the uninhabited type-formers (2856 of them), grades each candidate fix against the base verdict, and passes only when a fix flips the entire bug-set to REJECT (269 cases here) with no sound case newly rejected. No human labels these; the construction is the oracle.

The hard probes are different. Genuine runtime divergence is sound and must be preserved, so its golden is VERIFY, but base is not a usable reference there, and the gate's grammar contains no divergence-preserve shape, so the gate is blind to this arm. Its golden comes from human judgment; the two divergence probes are held out, authored by hand and kept outside the gate's grammar, so passing them tests whether a fix represents the rule rather than fits the gate. The two are not equally hard. One is in-bar: the maintainer's

approved fix preserves it, so its VERIFY golden is corroborated by the shipped human bar. The other is a stretch goal beyond that bar: force-graded, the maintainer's own fix declines it too, conservatively by design, so its VERIFY golden is the operator's ideal, not a behavior any shipped fix delivers. This is the same asymmetry the deployment story rests on (§7.5): the oracle for the easy arm is free, the oracle for the hard arm costs a human.

One integrity hazard is controlled before any verdict is trusted. The compiler vendors a crate that a plain incremental build does not rebuild, so every verdict here is from a forced-fresh build, sources touched and the binary fingerprinted to confirm identity; the frozen dataset (21 artifacts, every one rebuild-confirmed) and the regrade script are committed.

7.3 The lift: Verus #2219

The bug: an erased ghost expression of the never type `!` marks the following MIR unreachable, so a sound program is wrongly rejected. The narrow fix keys on `!`, which is what the maintainer's first patch (#2230) does. The general fix is harder: it has to keep the control-flow edge for a ghost-erased `!`-return while still pruning it for genuine runtime divergence. The maintainer's later patch (#2501) does this by recording each call's inhabitedness in Verus's own semantics and staying deliberately conservative, keeping the `is_never` test but gating it by call context rather than swapping in a broader type query, and it took deep Verus knowledge.

On a fixed toolchain with forced-fresh, fingerprinted rebuilds, six prompt-encoded methods across eighteen draws reached the narrow plateau and stopped: modal `changed`=114, the #2230 slice, none reaching `pass=true` (graph, minimal, neutral, site-enumeration, abduction, self-verifier). The single externalized-gate arm was the sole one to break off the narrow plateau to a wider fix: `pass=true`, `changed`=269 exactly, zero valid-preserve rejections on the gate's own cases. That fix is more general than the plateau but not yet the merged human fix; it stays wide-but-broken on the in-bar divergence case the gate never enumerated, the one the human fix preserves, dissected in §7.4.

What it does reach is real: it flips the entire bug-set, and it rejects two out-of-grammar held-outs the gate never showed it, an associated-type projection (`<u8 as Tr>::A`) and a nested generic (`G<G<Void>>`). Those held-outs are the proof that the fix represents a general rule rather than tabulating the gate: instrumentation confirms each arrives already normalized to an uninhabited type, so the fix catches them by applying its own general predicate, never having seen the case. Which predicate that is, and how it differs from the maintainer's, is the subject of §7.4.

7.4 The frontier is the coverage

The mechanism, read from the 43,586-line climb trace: The model's generalization frontier is set by the gate's coverage. Its first implementation is narrow, keyed on `is_never()`, the exact ceiling every prompt arm stops at. The gate then feeds it uninhabited cases that `is_never()` does not catch, empty enums and recursive-uninhabited constructors, which keep `mishandles>0` and bar `pass=true`. To climb, the model greps the codebase, finds rustc's existing inhabitedness machinery, and widens its predicate from `is_never()` to a check that keeps the CFG edge for any visibly-uninhabited ghost return, routed in this arm through rustc's `is_inhabited_from`. That widening off the narrow plateau is the lift, and the pressure that drives the model to it comes from the gate, while the prompt stays silent: the model supplies the discovery, the gate supplies the direction and the boundary.

A cross-model check narrows what the lift is. The inhabitedness query the codex arm routed through turns out to be behaviorally redundant: handed the same task, a second model (Fable) keeps the edge for every ghost-mode call with no inhabitedness query at all and grades identically on every probe, the out-of-grammar held-outs included. Since only an uninhabited return prunes the control-flow graph, the operative mechanism in both fixes is the same mode gate, keep the CFG edge in ghost mode so the borrow-checker sees the code that follows, and not recovery of the verifier's own decision procedure. So the lift is the model widening, under external pressure, to a mode-gated approximation of the right rule; it is not the model reaching the verifier's true oracle.

The handed gate's fix stays wide-but-broken here, over-rejecting the in-bar divergence case the merged human fix preserves, and the merged human fix reaches that case by a different, deliberately conservative implementation. That gap is not a permanent capability shortfall: once the missing calibration is supplied (§7.5), the corrected arm reaches the human fix's behavior on every graded probe. The held-outs still do their job: the fix catches `<u8 as Tr>::A` and `G<G<Void>>` it never saw, even though the rule it represents is the coarse mode gate, not the fine inhabitedness distinction.

The same mechanism predicts its own failure, which is why the failure belongs in the dissection and not in a caveat. The fix over-rejects two genuine-divergence cases, both sound. One is in-bar: the merged human fix preserves it, so over-rejecting it is the handed gate's real flaw. The other is a harder stretch case that the merged human fix also declines, conservatively and by design, because the bar it shipped does not promise it; over-rejecting that one is not a flaw the human fix avoids. The over-rejection sits exactly where the gate was blind: its 2856-case grammar contained no genuine-divergence-preserve shape, so that arm of the distinction received no climbing pressure. The target is a "fancy XOR": keep the edge for ghost-erased uninhabited returns, prune it for genuine runtime divergence. The gate drove the first arm to full generality and left the second collapsed to an OR. The fix is wide but broken, general on the uninhabited-return arm and over-conservative on the divergence arm, and the held-outs outside the gate are what catch it. Coverage is the design lever, shown from both sides in one trace: where the gate pushes, the model generalizes correctly; where the gate is silent, the model over-generalizes.

7.5 Enumeration is inducible; the oracle is not

The dissection has one more layer, and it locates the encoding boundary. Handed the same vague, leak-free prompt the prompt arms got, with no gate supplied, a second model reconstructed the gate on its own. It built a 7026-case enumeration over the uninhabited type-formers and hill-climbed against it to a self-certified zero over-rejections. Then it shipped the same wide-but-broken fix. A model can bootstrap the enumeration, the combinatorial breadth, because that is mechanical. It cannot bootstrap the oracle, the external ground truth, because its self-built gate labels each case with the very predicate under test, so the one case that needs a truth from outside its own belief, genuine divergence versus ghost-erasure, gets self-mislabeled as handled. One run shows both halves: a wide net the model built itself, and a blind spot precisely where its own labels could not reach. Enumeration is inducible; the oracle is not, at least not here: the label the hard arm turned on could not come from the model grading itself against the very predicate under test, and had to be delivered from outside. The claim is scoped to that circularity, not a proof that no model could ever induce an oracle from some other source. Within it sits the line between the reasoning a model already carries and the reasoning that must be delivered from outside.

A theoretical intuition sits under this, offered as motivation rather than proof. The data processing inequality (Cover & Thomas 1991) says no processing of what a system already holds can raise its information about the world; read as an analogy, enumeration is internal recombination and stays under that ceiling, while the oracle comes from a world-facing trial, the kind of step that admits information the weights did not contain. A model grading its own cases never runs that trial. Turning this into a theorem about LLM inference would mean formalizing the channel and the variables, which is out of scope here; we lean on it only as the shape of the boundary, the same place the receipt already locates it. (Compress and Unfold develops the same intuition from the cognition side.)

The half a model cannot induce is nonetheless cheap to the harness, because humans already rendered the judgments and left them in the repository. The merged fix, the regression suite, the resolved-issue label are goldens because a reviewer approved them. The missing oracle for #2219 existed all along: the maintainer's general fix verifies the divergence case the gate over-rejected, so calibrating differentially against base and the approved fix should close the blind spot, and supplying it does.

Handed that corrected calibration, the model crosses off the wide-but-broken plateau onto the divergence arm and writes a real ghost-versus-genuine discriminator. Force-graded against the merged human fix at its own toolchain, the corrected arm matches it on every probe: it now preserves the in-bar divergence case the handed gate over-rejected, and the one case it still declines is the stretch goal the human fix declines too. So calibration closes the gap to the shipped human bar rather than landing short of it; the residual sits beyond that bar, not below it. That the wall gives precisely when the missing oracle is supplied is the boundary's confirmation, it was the oracle and not a deeper limit, and the unblocking is model-dependent enough to be a property of the deployed workflow rather than a claim about weights, with implementation the residual wall where it does not occur.

The honesty here is an asymmetry, and it is structural rather than a hedge. Approved history supplies strong goldens for "do not break what works" and "this reported case is wrong", while the generalization itself goes ungraded, which look-alikes are the same bug and which are sound divergence. That disambiguation is the XOR's hard arm, un-oracled until a human spends the judgment, which is why the maintainer's fix took expertise. And the hardest look-alike is hard enough that the merged human fix declines it too, shipping a deliberately conservative bar rather than the full distinction, so past that point the wall is not the automation's alone.

Not one model's artifact. Under the same corrected-gate pressure, three workflows are driven to the same behavior, and force-graded against the merged human fix they match it on every probe: Fable, Sonnet 4.6, and Cursor's Composer 2.5, each on its own vendor CLI, clear the bug arm, preserve the in-bar divergence case, and decline the one stretch case the human fix also declines. The codex-CLI workflow does not clear it in either protocol, and a matched-budget, matched-protocol rerun leaves the divergence wall standing, so the gap is a workflow difference on this one case rather than a budget artifact; it is one n=1 cell per workflow and not a capability ranking.

Read the convergence as coverage-bound rather than as a scoreboard: it shows the corrected-gate fix is reproducible across workflows, not that three models independently rediscovered the predicate, since the shared decline on the stretch case is most likely the gate funnelling every successful arm into the one behavior it rewards, which the human fix happens to share.

Contamination is scoped the same way. Composer 2.5 ships after the fix, so its pass leaves recall not excluded; the two contamination-clean workflows (Fable, Sonnet 4.6) carry the narrower point that these runs do not require a recall explanation, and the merged human fix, dated after Composer's ship, is the anchor that makes that point datable. One earlier anti-recall argument is withdrawn here, because the control that would have carried it was finally run. The claim that the human fix is finer than the automated arms, and clears a case they miss, does not survive force-grading: the human fix is behaviorally identical to them on these probes and so cannot separate a memorizer from a reconstructor. That every successful run here uses the same externally supplied discriminator is consistent with a broader claim about how inquiry works, taken up as a conjecture in §12; the receipt here stays scoped to what these runs show.

Retrospectively the goldens are free; prospectively, on a fresh bug with no merged fix, the easy arm has a golden and the hard arm does not. The division of labor is the deployment design: the model enumerates and fixes, the harness draws its oracle from approved history, and the residual hard arm is named, not hidden.

The whole ablation reduces to one grid, and the grid has a structure: the two middle columns flip on as soon as the enumeration goes wide, and the last column flips back only at the human general fix. The first flip is enumeration, which an external gate supplies and a model can even rebuild for itself. The second flip is the oracle for the hard arm, which no automated arm here ever earned, whether its kills were graded against the base alone or self-graded by the model.

Arm	Kill oracle	chg	'!'-bug	empty-enum	out-of-grammar	divergence	Bucket
6 methods (18 draws)	self	~114	Y	N	N	Y	narrow
#2230 (maintainer, narrow)	human	114	Y	N	N	Y	narrow
`abductor` gate (handed)	external (vs base)	269	Y	Y	Y	N	wide-but-broken
self-built gate (induced)	self (induced)	269	Y	Y	Y	N	wide-but-broken
#2501 (maintainer, general)	human	269	Y	Y	Y	Y	general

The Verus #2219 ablation, one row per arm, Y where the arm handles the probe correctly (rejects the bug cases, preserves the sound ones). The reported '!'-bug column is solved by everything. The two middle columns, in-grammar empty-enum and the out-of-grammar held-outs the gate never saw, are the uninhabited-return generalization; they flip on the moment the enumeration goes wide, for the gate graded against an external oracle and for the gate a model induced for itself alike, and stay off for the narrow arms. The last column, the in-bar genuine divergence that must be preserved, is the XOR's hard arm; it is on for the narrow arms (which never reach far enough to break it) and for the merged human fix, force-graded to preserve it, and off for both wide gates, which over-reject it. (A harder divergence stretch case, beyond what the merged human fix promises, is declined even by that fix and is not a column here.) Enumeration moves an arm from narrow to wide and is inducible; the wide gates shown here, graded against the base alone or self-graded by

the model, do not on their own preserve the in-bar divergence case. Supplying the divergence golden is what later moves a corrected arm onto it, to the human fix's behavior (§7.5).

7.6 Two surrounding cases, briefly

Verus #2219 is the case under study; two earlier cases bound it and are kept only as pointers. flux #1613, an open, maintainer-stuck refinement-typing bug, is retained only as an auditability example, a 19-node replayable trail at tag `flux-1613-trail-v1` that stops short of a capability lift: de-hinted and run paired, its advantage oscillates between general and narrow rather than holding. Verus #2427 marks the boundary from the other side: the external oracle was in place and did not engage, because the inquiry's perturbations stayed inside one feature family and never crossed the dimension separating the narrow fix from the general one. Together they fix the band the lift lives in, as one rule: an external oracle lifts capability when, and only as far as, the inquiry's perturbations span the dimension that distinguishes the reported example from the sound generalization. Both cases are in the repository for any reader who wants the full trails.

7.7 The null regime, and why it is narrow

The lift engages in a narrow band, and most pilots fall outside it. On bugs drawn from the merged-PR pool and beyond it, the minimal baseline kept succeeding: a 5-line CLI fix in 91 seconds (qrtool #695), the deepest graph in the pool out-reasoned in two minutes (slang-server #310), a real clear()/ingestion data race in an LSM storage engine the pipeline had skipped as too complex (fjall #287), a decoration-leak the minimal agent localized unaided (bat #3710).

Two findings explain the nulls. First, a selection artifact: the deployment pipeline's own triage skill scores reproducibility up and fast-paths easy bugs around the graph (its rule, verbatim: "a 1-line fix with a confirmed reproducer doesn't need a hypothesis graph"), so the merged pool is precisely the subset where the pipeline says the graph is unneeded. Second, baseline reach: a capable model in a minimal loop diagnoses and fixes most reproducible bugs unaided, so the band where diagnostic structure changes a pass/fail verdict is narrow, bounded below by the minimal loop's own reach and above by triage.

The deployment merge rate inherits this: the 81 merged PRs (§8) were never graph-dependent, a minimal prompt digs the few levels most of them need. The Verus case is chosen to sit inside the band, where the symptom is far from the cause and the suite cannot see the difference between a narrow fix and a general one; that placement is the point, and it is also why a population rate over an unselected pool would measure the band's width rather than the mechanism.

8 The bench, bounded

Does the method improve benchmark performance? The obvious question of any harness, and it does not apply here. The assembly was run on SWE-bench Pro and resolved 95.3% of the public split under the official grader, but that number is not a property of the harness or the method. It is a property of oracle availability: on the public split the failing tests are visible, the gate iterates against them, and any competent coding agent handed that signal reaches the mid-nineties. The oracle bracket prices it exactly (§8.5). Leading with this number was a mistake; correcting it is why the bench is cornered here, after the result that bears on the method.

There is a deeper reason the question misfires. A benchmark grades performance, and performance is not where a harness is bottlenecked. What a harness struggles to deliver is observability, reliability, and attribution: whether you can see why it acted, whether the result holds without re-deriving it, and whether the credit can be pinned. A score reports none of those, and the four problems opened here are all of that kind, not one of them a shortfall in raw capability. SWE-Effi reaches the same place from the model side: once the gate and its oracle are fixed, neither scaffold typing nor model tier moves the number more than about two points (§10.2). So the handover is only half of it: what a bench can measure is no longer what a harness most needs to get right.

Every choice follows one goal: making the run attributable, so a skeptic can pin the result on the harness or rule it out. The disciplines live as receipts in the repository rather than as prose here: preregistration and an annotated freeze tag (`PREREGISTRATION.md`), eligibility against documented bench defects (`KNOWN_BAD.md`), official-grader-only verdicts, infrastructure fault classes predeclared with invariants so recovery cannot become a re-roll lever, per-instance provenance (trajectory, hypothesis graph, captured diff, gate trace, cost ledger), and a doubt-by-doubt guide for hostile readers (`FOR_SKEPTICS.md`, `OBJECTIONS.md`). The operational story, including everything that went wrong on the way to an honest number, is the companion field guide How Not to Run SWE-bench Pro.

8.1 Models and the oracle boundary

No training, no fine-tuning, no learned weights in the harness. Each stage invokes a vendor's shipped agentic CLI (Claude Code for the Sonnet 4.5 generator, codex for the GPT-5.5 challenger, Cursor in the open-weight run), so the harness is a typed meta-loop over off-the-shelf agents, owning only the stage contracts, the hygraph, and the gate between them. The generator ran with extended thinking on; the challenger arbitrated with reasoning off, a point against a model-deliberation reading of the gate. For each scored instance the loop reads only that instance's own artifacts; no other instance's graphs, trajectories, or solutions enter the context.

The boundary that matters most: on the public split the bench's `FAIL_TO_PASS` tests are visible, and the frozen harness's in-container gate executes them as its stop signal (their names as budget control, their bodies never in the model's prompt). The standardized leaderboard scaffold is denied them by design. That difference is the dominant attribution channel (§8.5), and the held-out split removes even the names, so nothing in the public-split numbers here transfers to the private set. Swapping the entire model pair to open weights ported the result with no structural change, evidence the harness is not vendor-specific; that run's near-gold tail discounts as recall, with the details in the repo.

8.2 The number, with receipts

The texture behind the rate is committed for inspection, the receipts in place of narration: the whole eligible public set graded with 0 incomplete at frozen tags `prereg-pro-v1` and `prereg-pro-v1-cheap`, per-repo tables (ten of eleven repos above 92%), the first-pass/re-entry split (~93% of trajectory-captured wins resolve on the first pass), the development-overfit check (the development language resolves lower than the never-touched languages), re-grades that reproduce (6/6 frontier cross-language, 60/60 open-weight stratified, 0 flips, ~$3 per WIN to audit), and all 34 losses carrying non-empty rejected patches. Start at the repository scoreboard (github.com/kimjune01/swebench-pro).

OSS deployment trace. 81 PRs merged across 73 cold repositories at a 50.6% merge rate under adversarial maintainer grading: agent-selected issues, agent-authored patches, agent-submitted PRs, zero human keystrokes in any diff, and only ~8 of 79 closures rejections on the merits. The ledger stays GraphQL-verifiable (`pr-receipts.jsonl`); ~385 hypothesis graphs from the same campaign are public at `kimjune01/sweep/repo-hypotheses/`; the narrative is Speedrunning Open Source.

Two receipts, two independent attestors:

Receipt	Evidence	Attestor
Pro (preregistered)	terminal: 694 / 728 = 95.3%, 0 incomplete, whole eligible set graded	Scale's official grader, re-runs the committed patch
OSS PR merge rate	81 merged across 73 cold repos, 50.6%, GraphQL-verifiable	adversarial maintainers who merged

We do not attest; the receipts do. The contribution is auditability, not the rate: no method documented in our comparative search (§15) publishes per-instance receipts at this depth on SWE-bench Pro, official-grader-only, whole eligible set, zero incomplete. That claim is about the artifact, and it holds whatever the rate turns out to be. The rate itself answers nothing claimed here, for the reason the attribution below makes exact.

8.3 Three limits on the number

Three readings the number does not support, stated before the attribution rather than after:

- Not a leaderboard number. The board ranks models through one fixed scaffold; this run holds the model roughly fixed and rebuilds the scaffold. A harness number has no slot on a model board by construction.
- Not a held-out number. The public split is the audition; Pro's private split removes even the test names, so the gate must go blind there, and nothing here predicts that result.
- Not an isolation of the method. The within-pass `implement`->gate refinement loop, the vendor inner agents, the thinking-on generator config, and the gate's oracle access all ride along. The next section separates them.

8.4 Attribution: where the lift lives

The attribution decomposes the gap channel by channel. Each cut is preregistered or receipt-committed in the repository, including one retracted estimate and its worklog trail; we keep the findings here.

Channel	Cut	Effect on Pro resolve
Gate access to the visible tests	oracle bracket, n=50	~46 points (50% floor -> 96% ceiling)
Peircean prompt vocabulary	M vs G vs T, n=38/36	null (CIs straddle zero)
Directed diagnostic perturbation	deprived arm, n=110	+0.105 on underdetermined-cause stratum only; threshold-level
Entire diagnosis stage (the smem)	minimal-prompt arm, n=34	~1 point, resting on two instances
Model pair (frontier -> open-weight)	pair swap, n=728	2.2 points raw; ~17–22 genuine after recall discount

8.5 The oracle bracket

The largest channel is the one the original design held constant and therefore could not see. On a preregistered 50-instance random sample, an implement-only loop with no oracle access floors at 50%; restoring the bench's tests for the gate to iterate against raises the ceiling to 96% (`PREREGISTRATION-baseline-replication.md`, the bracket). Forty-six points sit between those arms, bought by oracle access, with the models reasoning no harder than before; the floor sits near the bare-model board scores (43.6–64.3%), the ceiling at the headline. The companion posts reached this conclusion first (Precisely Wrong, How Not to Run SWE-bench Pro).

8.6 Given the oracle, the rest adds almost nothing

Measured against that oracle, every remaining channel barely moves, and they tell one story. The methodeutic prompt vocabulary scores no better than generic rigor or the bare task (null, CIs straddle zero). Deleting the whole diagnosis stage, a minimal prompt with no graph handed only the problem and the failing tests, lands near 94%, about a point under the full harness: the minimal prompt reaching the number is the attribution made plain. The aimed diagnostic probe buys only a threshold-level sliver, and only where the text does not determine the cause, because the gate's cheap, trustworthy verdict lets blind try-and-check substitute for aimed diagnosis; that substitution is the falsifiable handle, degrade the gate and the deprived arm should worsen, which is the regime the mechanism experiment runs (§7). The model pair, the lever leaderboards rank, moves the raw rate two points. Therefore the answer to how well does the method work is: not on this bench, because the bottleneck here is not diagnosis. The full per-channel statistics, one retracted estimate and its worklog trail included, live in the repository (swebench-pro).

8.7 The benchmark gap

The deeper reason the number says nothing about the method is an audit finding, not a starting premise. A determinacy audit of the task set found the requirement handed over and the cause already named: SWE-bench Pro measures spec-conformance, so the diagnosis is presented as part of the problem, and a tool whose job is diagnosis has nothing left to do. The bench is the wrong instrument for a diagnostic method, sound in itself but misapplied, a question of fit. That finding is what sent the inquiry to harder problems, where the cause is genuinely hidden and no spec is handed over, and §7 is the case that search found. The nulls of §8.4 are not the method failing; they are the predicted signature of a mechanism that fires only where the bench cannot reach.

9 Discussion

9.1 Attributed nulls and the typing protocol

The discipline that bounds the bench number (§8) is the same one that makes a null publishable: preregistration, freeze, official-grader-only verdicts, per-instance provenance. These nulls publish attributed, each arriving with the mechanism that produced it, the gate compensating, the spec handed over, the lottery fraction. An unexplained null says stop; an attributed null says where to point the next instrument.

The same typing runs on the positive claims, not only the nulls. Each result is held to the verb its evidence can carry: the Verus case is a mechanism that can occur, shown by one dissected inquiry, not a rate and not a discovery in the sense of a frequency, and saying so is the typing protocol applied to ourselves. This is the Type III check, the wrong question caught before the claim is committed, and the hygraph already encodes both of its corrections: a kill condition is a reflex, a mechanical check run before a belief is granted, and the credence cap is a demotion, the narrower claim a node settles for once the evidence is in.

Does the Peircean framing do any real work, or is it just decoration? Two kinds of work: on this bench, as rhetoric it is decoration (§8.6); as protocol it is load-bearing. The prompt ablation measured whether the vocabulary makes a single diagnosis smarter, and it does not. What it could not measure is interop. A hygraph written in one context window must be read in another, by a different model, a parallel agent, or a human auditor, and the mode labels are what make a node interpretable without the context that birthed it: abduction marks a belief as proposed-untested, induction marks it as test-backed, and the credence cap travels with the type. Loose vocabulary produces graphs only their author's window can interpret, and those die with it. The Peircean typing is the wire format that lets the smem be shared across compactions, across agents, and across the trust boundary to the auditor, and the common vocabulary is the protocol for verifying each other's work: a peer that reads `induction, test-backed, here is the command` can rerun the command, where a peer that reads loose prose can only believe it or not.

This resolves the apparent tension with the prompt null rather than sitting beside it. A protocol does nothing on its own on any individual instance, the same way a wire format makes no single message smarter, so a per-instance ablation measuring the vocabulary against loose prose should read null, and did. The protocol's value accrues across instances, where accountability becomes transitive. When every node speaks the same typed contract, agent B can verify agent A's kill by rerunning it, agent C can build on B's verification without re-running A, and a human auditor can enter the chain at any link; the warrant flows down the chain with the receipts, never resting on any link's word. Loose vocabulary caps accountability at one hop, the author vouching for their own graph. A common protocol lets verification compose: that is what 385 graphs in one vocabulary are for, and the precondition for everything §12 builds on them.

9.2 The trace is the contribution

The reframe is from treating an agent as a source of verdicts to treating it as a source of claims bound to evidence.

The failure mode at issue is output that is confidently wrong and not cheaply verifiable. The Verus self-built gate is an instance of it. Handed a vague prompt, the model built its own 7026-case generator and hill-climbed to a self-certified zero over-rejections, reported the fix done, and shipped one that over-rejects genuine divergence anyway: green by the agent's own check and unsound at once. Nothing the gate enumerated detects it; only the held-out probes outside the gate, replayed against the patched build, expose it. This failure mode scales with model capability rather than against it, and is the default product of fluent generation; it recurred throughout this work, in the wide-but-broken fix, the 4/4 false-green self-audit, and the recall-inflated resolve rate.

The substitution asked of the reader here is accountability in place of trust, checkable line by line. The self-graded agent produced a verdict, its own gate green; the externally-oracled inquiry produced a record of typed nodes, each a hypothesis, an exact command, an observed outcome, and the edge the result generates, every load-bearing one replaying on a clean build. Such a record is audited step by step without extending trust to its author, and the better fix is identifiable independent of the system that produced it. That property, rather than the outcome of any single comparison, is what survives.

Truth is buildable. The node semantics of §3 is what gives the trace its force here. If a true claim is a build presently passing and its warrant lives in the edges, then a node without a replayable trial is not a node at all, as an uncheckable number is not a measurement. A resolve rate, however honest, is a verdict over 728 builds the reader cannot inspect; a hygraph is the inspectable chain.

The cost asymmetry of fabrication. A fabricated reasoning trace is expensive to sustain, because every fabricated node must survive a replay the author does not control. A confident narrative is cheap to produce, because nothing in it is bound to a procedure. A self-graded method's over-narrow fix is the cheap kind: it reads as complete, and closer reading does not reveal the flaw, while the receipt reveals it in one command. Verification is therefore part of the method rather than overhead on it. The same asymmetry governs failure: a trusted verdict that fails leaves nothing behind, whereas a recorded inquiry that fails leaves a trail naming the failed node. The Verus climb recorded its own corrections in trail, the narrow `is_never` fix refuted by the very cases the gate fed it, each a kill that generated the next edge and itself a replayable trial.

Persistence beyond the context window. The context window is the model's working memory, and chain-of-thought stored there is lost when it closes. The hygraph externalizes the reasoning before the window closes, demonstrated when a build host failed mid-investigation and only the externalized graph and patch survived to be resumed. The externalized form is not merely readable later but re-runnable later, provided each node carries its own reconstitution; without that, it only appears to outlast the window until resumption is attempted.

The property also runs forward in time: trust weakens as witnesses are lost, whereas a replayable trial copies without loss and persists as long as one copy does.

The scope is bounded: the logical content has no half-life, but the runnable form inherits the half-life of its apparatus (these graphs replay only while flux builds, z3 runs, and the SHAs resolve), so provenance approaches durability as replay approaches first principles. What it preserves is checkability rather than truth: a false claim backed by trust is laundered as its witnesses disappear, while a false claim backed by provenance remains detectable.

Hidden effort is not reasoning you can check. The frontier labs ship "reasoning" as an opaque dial, `high`, `xhigh`, `ultrathink`, a slider on how many private tokens the model burns before it answers. More of that setting is more internal work: the model proposes and grades against its own belief, harder.

The Verus result is where that runs out. The strongest internal-effort arm, a self-verifier told to build its own combinatorial generator and grade itself against it, plateaus on the narrow case alongside every other self-graded method, while a single kill graded by an external oracle reaches the general fix (§7.5). You cannot turn the dial past this boundary, because effort scales the half that is inducible, the enumeration, and cannot manufacture the half that is not, a ground-truth oracle from outside the model's belief. And whatever the dial does buy ships nothing a stranger can rerun: an effort setting is the purest form of take my word for it, a larger number where a kill condition should be.

The position here is the opposite. Put the reasoning in the harness, typed and replayable, where its level is not a knob the reader trusts but a trail the reader checks. A graph of fired kills says what was ruled out and how; `ultrathink` says only that the model tried hard, and asks to be believed.

Trust versus accountability. Nullius in verba extends to the machine: take no party's word, and check the receipts. To a maintainer a coding agent is a stranger, as is its operator, and with receipts attached this does not matter: a red-on-master/green-with-fix test, a soundness twin, and a clean suite read identically regardless of submitter. The 81 merged PRs are the ecological evidence: adversarial maintainers accepted agent-authored code at 50.6% on attached evidence rather than reputation.

The maintainer evidence points at what alignment can mean: an accountable agent whose every claim is bound to a test a hostile party can rerun, not a trustworthy agent that is then believed. A recent DeepMind position paper on artificial epistemic agents warns that verification by source authority is collapsing into a "verification crisis" and calls for "robust falsifiability pipelines" whose claims are "structured in a way that allows them to be proven wrong" (Marchal et al. 2026, arXiv:2603.02960); the kill condition is precisely what structures a claim to be provable wrong. Reliability is this same epistemics accumulated: a body of attestations large and redundant enough to remove doubt, rather than trust arrived at directly. The guarantee is narrow: everything attested is checkable, but nothing guarantees that everything relevant is attested, and choosing what must be on the ledger remains a human judgment.

The equilibrium resembles accounting controls. Alignment is unlikely to resolve to a single answer. A plausible regime has the shape accounting reached after its own confidently-wrong-and-unverifiable era: standards (GAAP), independent audit, and the three-way match, in which no payment clears unless the purchase order, the receiving report, and the invoice agree, with no party trusted to hold more than one leg. This harness already has that shape: no claim clears unless the proposer's claim, the recorded trial, and an independent replay agree, and the generator never grades its own work. Enron illustrates the alternative: self-attestation scales until the failure that legislates the controls after the fact.

Merit attaches to the work, not the doer. Praise and blame are routed to doers for lack of vocabulary and norms that would route them to the work, a conflation that was affordable while only humans produced work, since the doer was a usable proxy for its quality. It is no longer affordable. An agent can produce many artifacts of any quality, so judging them by their author is backwards, and judging the author by replaying the artifacts is the only direction that scales. Merit is the warrant a piece of work carries in itself, checkable without reference to who or what produced it. The hygraph is a unit of work that ships with its own evidence; the receipts-first PR is the corresponding practice; the maintainer who merges on the ledger alone is an early instance. The doer earns standing only as the accumulation of work that has survived such checking.

Legibility. A benchmark number is legible to readers who will never read a trace, and that reach has value; it is how this work reached an audience. But the number is the artifact a reader can do least with: it cannot be perturbed, replayed, or used to generate a rival hypothesis. The Verus trail can be: a reader who suspects the model reached the general predicate by luck can replay the climb, construct a new discriminator off the

gate, and attack a node. A benchmark result is an answer; a replayable example is an instrument, and instruments are what an inquiry accumulates.

9.3 What the structure unlocks

A data structure earns its keep by what it unlocks. Four affordances follow, in ascending order of consequence.

Parallel agents, shorter wall-clock. The graph is monotone: nodes append, kills are idempotent. Parallel agents can therefore latch onto one shared graph lock-free, fan out across rival hypotheses, and re-verify each other's kills instead of trusting them, with transitive accountability (above) as the precondition that makes the fan-out safe. A prose summary must be re-parsed into independent units before anything can be dispatched; the graph ships pre-factored. Named, not yet run (§12).

Model-provider independence. The smem lives in the harness, in plain markdown, behind typed contracts any capable model can read and write. The pair-swap run is the witness: the entire model pair changed and the structure ported wholesale (§8.1). Reasoning that accumulates in a vendor's context window is a liability; reasoning that accumulates in a substrate you own is an asset.

Accountability. Every claim ships with its replay.

Alignment. Trust displaced by audit: an agent does not need to be believed to be used.

10 Related work

10.1 Construct validity and contamination

The SWE-bench family defines the Verified / Pro lineage, official harness, and contamination-resistant tier design. SWE-Bench+ (Aleithan et al. 2024) manually audited the original bench: 32.67% solution leakage, 31% weak tests. OpenAI's February 2026 audit found a majority of audited Verified tasks have flawed tests and that frontier models reproduce exact gold patches; it stopped reporting Verified and recommends Pro. Wang, Pradel & Liu (ICSE 2026) show plausible patches pass tests yet diverge from developer intent; their axis is patches that pass but are wrong, ours (§8.7) is tasks whose materials do not determine which passing behavior is intended. ORACLE-SWE (arXiv:2604.07789) quantifies the same handover, ablating the oracle and specification signals that leak through a task and measuring the resulting drop; SLUMP (arXiv:2603.17104) opens on the identical premise, that benchmarks supply the full specification upfront while real coding does not, and answers it by building an underspecified-by-design benchmark. Here we part from both on the verdict: they treat handover as a defect to fix with a better benchmark, while the determinacy audit (§8.7) draws it as a category boundary, a spec-conformance instrument cannot be tuned into a measure of diagnostic inquiry because the two are different types. SWE-rebench uses post-cutoff filtering as a parallel contamination strategy; LiveCodeBench (Jain et al. 2024) is the origin of post-cutoff (temporal-holdout) evaluation, and the standard objection to it applies here too, that training cutoffs are porous because RL post-training and inference-time retrieval can surface later content. The witnessed case (§7.2) is built against exactly that objection: it is post-cutoff and the solving model's weights predate the fix, so neither porosity nor retrieval supplies the answer. HAL (Stroebl et al. 2025) is the third-party cost-aware agent leaderboard and the nearest infrastructural precedent for the cost-transparency stance here; the receipts here go one level finer, to the per-instance re-gradeable verdict. The official swe-bench/experiments repo requires `trajs/`, `logs/`, `patch.diff`, `report.json` per submitted instance, the minimum publication norm our provenance contract extends with gate traces, hypothesis graphs, and a cost ledger.

10.2 Agent scaffolds and SE-agent harnesses

SWE-bench-targeted harnesses include OpenHands (Wang et al. 2024), SWE-agent (Yang et al. 2024), and AutoCodeRover (Zhang et al. 2024/25), all built on the ReAct pattern (Yao et al. 2023); none implements Peirce-typed stage contracts or a kill-conditioned hypothesis-graph memory. Voyager (Wang et al. 2023) is the closest loop-shape precedent: embodied observe->hypothesize->test->commit, with a skill library where this work holds falsifiable claims. SWE-Effi (arXiv:2509.09853) is the sharpest published counter-position: effectiveness emerges from scaffold-model synergy rather than residing in the scaffold alone. Here we agree from the other direction, with the synergy named: on Pro the binding pair is gate × oracle, and neither scaffold typing nor model tier moves the number more than about two points once that pair is in place.

Two concurrent developments arrived independently at adjacent points, each carrying one of the two components composed here. Theorem-of-Thought (Abdaljalil et al. 2025) types reasoning into abductive, deductive, and inductive specialist agents per query: the typed cycle, without a persistent typed memory across cycles. Cognitive Memory Manager (Khalid & Arora 2026) extracts a typed-node DAG by observing agent execution and mines it for patterns to promote to skills: the typed graph, mined descriptively where ours is generative

(it routes the run). That convergence is independent, not coordinated. Provenance for the framing here is timestamped on the project blog (The Hypothesis Graph, Evidence has a trajectory).

Convergence forces a vocabulary question, and we answer it here by pointing rather than claiming. The trichotomy the siblings reach for is Peirce's (1878, 1903). Theorem-of-Thought builds abductive, deductive, and inductive agents while citing no pragmatist anywhere in its reference list (checked against its v2 source: forty-four uses of the mode words, zero occurrences of Peirce, pragmatism, James, Dewey, or Ramsey), so the typing in that line of work is used without its source citation. §3 and §16 wire the vocabulary to its sources, and the dated posts above timestamp this lineage's use of the hypothesis-graph primitive. Interop (§9) will force one wire vocabulary on this design space, and the candidate is already to hand: mode-complete, a century and a half stable, and already carrying the credence semantics the nodes need.

System	Domain	Reasoning-mode typing	Persistent structure & update	Termination gate
Voyager (Wang et al. 2023)	Minecraft	None	Skill library; test-validated graduation	Test-pass on skill
IDEA (He et al. 2025)	Interactive rule learning	Peirce-cited, agent-level	Working rule set	None explicit
ADI (Gilda & Gilda 2026)	Algebraic invariants	Peirce, layered (L0/L1/L2)	Symbolic knowledge graph	None explicit
AriGraph (Anokhin et al. 2024)	TextWorld	None	Knowledge graph (entities, relations, episodes)	None explicit
CausaLab (Yang et al. 2026)	Causal discovery	Causal-typed (SCM)	Evolving structural causal model in a DSL	None explicit
BeliefMem (Liao et al. 2026)	Partial-observability QA	None	Candidate set; Noisy-OR probabilistic update	Probabilistic threshold
Theorem-of-Thought (Abdaljalil et al. 2025)	General reasoning	Abduction/deduction/induction, agent-level (no Peirce cite)	Formal reasoning graph	NLI-guided Bayesian coherence
CMM (Khalid & Arora 2026)	SE (coding agents)	7 trajectory roles, extraction-time	Typed DAG; confidence decay	Human approval + retrieval-validated threshold
This work	SE (industrial code)	Peirce, enforced at write time per stage	Hypothesis graph; mechanical kill predicates on the audit verdict	Deterministic finite-state

*1.** Comparison spine for adjacent typed-reasoning and graph-memory LLM-agent systems. Cell terseness is by design; prose nuance in §10.3.*

10.3 Typed reasoning and graph-structured memory

IDEA (He et al. 2025, ACL Findings, arXiv:2408.10455) explicitly cites Peirce and uses the three modes in an interactive rule-learning benchmark. ADI (Gilda & Gilda 2026, arXiv:2604.15727) gives an explicit Peircean tripartite protocol with epistemic layers over a symbolic knowledge graph; near-simultaneous with this draft and the most conceptually adjacent prior work. Both target reasoning domains outside SE.

The hygraph sits at the intersection of three lineages: cognitive-architecture memory (Soar / ACT-R / EPIC), LLM-agent memory systems (CoALA / AriGraph / Mem0 / Zep), and typed-belief representations (CausaLab / BeliefMem / Theorem-of-Thought / CMM). The hypothesis graph adopts the Soar memory typology directly as its slot vocabulary, adding only the specific content of the `smem` slot: Peirce-typed, kill-conditioned, designed for LLM prose read/write. Adjacent work: Kirk, Wray & Laird 2023 (AAAI), an LLM-port of the Soar lineage; CoALA (Sumers et al. 2023/24, arXiv:2309.02427); AriGraph (Anokhin et al. 2024/25, arXiv:2407.04363), the closest precedent for graph-structured LLM-agent memory; CausaLab (Yang et al. 2026, arXiv:2605.26029); BeliefMem (Liao et al. 2026, arXiv:2605.05583), strong adjacent on uncertain alternatives with mechanical update.

CMM (Khalid & Arora 2026, OpenReview; published one day before this draft) warrants its own comparison. Both systems converge on the same role for memory: a persistent, typed, queryable DAG of

reasoning artifacts. The runtimes diverge on agency. CMM is observe-and-consume: an external agent perturbs, CMM types the trajectory post hoc, future runs consume graduated skills. Our loop is perturb-and-falsify: kills fire mechanically during the live inquiry, and the graph routes the run. Same data structure, opposite epistemological direction, and complementary by construction: the ~385 graphs committed in `sweep/repo-hypotheses/` are exactly the corpus CMM's graduation pipeline could consolidate into per-repo skills.

Four 2026 systems each carry one component this work combines, which sharpens what is novel here: the join, not any single piece. FVDebug (arXiv:2510.15906) builds an actual hypothesis graph for debugging, with a frontier and accumulated evidence, but selects the next node by asking the model, the arbiter this work removes. From Hypotheses to Factors (arXiv:2604.26747) runs the same perturb-and-falsify loop, falsifiable hypotheses behind a deterministic engine over an append-only trace, locked to quantitative finance where this work claims the general semantic-memory substrate. Portable Agent Memory (arXiv:2605.11032) is the nearest provenance memory, a Merkle-DAG that cites the Soar lineage and makes every node reconstructible by content-addressing, but it certifies integrity (the recorded bytes are untampered) where the replay invariant here certifies warrant (the node still survives its trial). And the provenance survey From Agent Traces to Trust (arXiv:2606.04990) enumerates exactly the relations this work mechanizes, Support, Contradict, Invalidate, and names "how provenance quality should be evaluated" as an open problem; the hygraph is one answer, with replay as the quality bar and the kill condition as an executable edge rather than a descriptive label.

What the mechanism experiment (§7) adds to this comparison is which piece is load-bearing: not the graph structure, which several of these share, but where a kill edge gets its ground truth. FVDebug lets the model arbitrate the next step; the contrast that separates a narrow fix from a general one is whether the kill is graded against the model's own belief or against an external oracle, and the externalized form is released as `abductor` (enumerate a case space, calibrate against a known-good reference, gate). A model can rebuild the enumeration for itself but not that oracle (§7.5), which is the axis these neighbors leave implicit.

A second cluster of neighbors shares this work's epistemological ambition, treating truth and uncertainty as first-class rather than a downstream score, and locating where each stops marks what is claimed here. These are distinct from the cognitive-architecture memory systems above (Soar, CoALA, AriGraph): the question there is how an agent stores reasoning, the question here is what a stored claim means and how its truth is checked. NARS (Wang, the Non-Axiomatic Reasoning System) is the nearest cognitive architecture: an experience-grounded logic whose truth values are degrees revised by evidence rather than asserted from axioms. It holds graded belief as this work does, but its truth is a number attached to a term, where this work binds warrant to a replayable trial, and there is no provenance graph, no executable falsification edge, no trial a distrusting party reruns, and no proven / refuted / open ledger. OpenCog's AtomSpace and PLN give a typed hypergraph with truth values and executable procedures over it, but truth there is a label computed on the graph where here it is a build a stranger re-executes. Nanopublications (Groth et al. 2010) make a claim machine-readable and attach provenance, but the evidence is descriptive rather than executable, with no credence cap, kill edge, or replay. Closest in spirit and in time is Traxia (arXiv:2606.08256), an agent-native scientific-publishing layer with confidence intervals, signed identities, provenance, and a replication record over a living knowledge graph; it appeared two days after Truth Is Buildable and is a fourth independent group converging on these primitives, the first on the epistemology rather than the typing. It parts from this work on object: publishing infrastructure for findings, not the typed semantics of an agent's own memory, with no stakes-thresholded knowledge predicate and no falsifiability-as-kill-edge. The exposed flank across all four is the synthesis charge, that this is NARS plus a provenance log plus an executable-research compendium. The reply is that none of them makes the combination the semantic contract of a memory node, where truth is operationalized by replayable edge structure rather than a stored label or a textual provenance record, and the three states are an entitlement ledger, an account of what each claim has earned, not a new logic proposing a third truth value.

> Production LLM memory systems with graph variants (Zep/Graphiti, Mem0), staged-hypothesis selection in science agents, deterministic gating in adjacent settings, and reflective memory systems (Reflexion, DebugMate) are surveyed in the appendix; they are adjacent on particular axes but do not change the comparison spine.

10.4 Adversarial filtering and termination

System	Domain	Stage operated at	Visibility regime	Cross-family
Multi-Agent Debate (Liang et al. 2023/24, arXiv:2305.19118)	General reasoning	Patch / answer stage	Open (cross-visibility)	Single model family
Refute-or-Promote (Agarwal 2026, arXiv:2604.19049)	Defect discovery	Review stage	Asymmetric context	Yes
This work	SE (industrial code)	Pre-patch hypothesis stage	Blind challenge (no cross-visibility)	Yes (Sonnet + GPT-5.5)

*2.** Adversarial multi-model filtering: this work occupies the pre-patch / blind cell. Termination disciplines (_A's type-theoretic proofs, SafetyDrift's absorbing states) sit at composition or trajectory scope where this work's verdict-routed gate sits per-instance.*

Closest in spirit is POPPER (arXiv:2502.09858), which runs agentic sequential hypothesis tests under e-value error control, the same sequential-testing machinery this project's `inquire` workflow uses to classify evidence. POPPER terminates statistically, on an error-rate bound over a population of tests; this work terminates mechanically, on a deterministic kill predicate over a single binary verdict, and persists the outcome as replayable memory where POPPER's tests are ephemeral.

11 Limitations

The mechanism evidence is existence-grade. One audited divergence, on one instance, in a program that was not preregistered when it ran. The pilots' nulls are confounded by a selection artifact we can name but not yet remove (the triage fast-path), and the localization-hard band where the mechanism should live has not been decisively tested; its one strong candidate did not reproduce at HEAD. Nothing here is a rate.

The audit's two tiers carry different burdens. The mechanical spine (11.4%) is re-derivable by grep; the two-expert tier rests on a stated standard, adversarially verified ($= 0.52$, all disagreement skeptic-stricter), and 63 screen-flagged candidates are excluded as rater-pending. The proven floor is a floor.

The bench numbers are public-split numbers. Pro's public repos predate both model families' cutoffs; the gold-overlap audit bounds frontier reproduce-gold at ~2%, but our holdout is weaker than Scale's (different commits, same repos), and the held-out submission has not been made. The gate's oracle access does not exist on the private split.

Essence oracles are authored. The mechanism experiment's graders are written from upstream issue text by the operator's pipeline, mitigated by red-at-base/green-on-gold walls and by the merged fix's external attestation, and reduced but still present.

The smem is small and per-instance. Hypothesis graphs in this work are one markdown file per inquiry; cross-instance accumulation is untested. That carries its own deflationary point: the file was never the bottleneck at any repo size, so heavier stores need to earn their keep at cross-instance scale, where the per-instance case never demanded them.

How to refute this. The central claims are built to fail loudly, each against a committed artifact a hostile reader runs rather than a promise.

- The mechanism claim dies if the Verus receipt fails to replay, or if a discriminating program shows the externally-oracled fix wrong where the maintainer's general fix is right. The clean, forced-fresh dataset and both patches are committed to check, and the flux trail at `flux-1613-trail-v1` is there for the same test on the auditability case.
- The encoding-boundary claim, that the oracle is not self-generable, dies if a self-graded model reaches the hard arm with labels it authored independently of the predicate under test. That would show the oracle is inducible after all, and closing the gate's coverage on that arm is the next experiment's preregistered target.
- The methodology claim dies if, as audited cases accumulate, an external oracle stops separating from self-grading everywhere a receipt can see. The oracle's source is then not the active ingredient, and that null is named as this thesis's own falsifier in the mechanism README.
- The attribution dies if the oracle bracket fails to replicate from its preregistered sample.

Generator staleness. The checkpoints are fixed (Sonnet 4.5 era), and a more capable generator narrows the band where the mechanism is observable, since it resolves more cases unaided. The mechanism claim therefore survives only in the regime where verification, not generation, is the bottleneck, which is also the regime the discussion argues matters.

12 Future work

The program reorganizes around the smem, in order of leverage.

- The preregistered mechanism hunt. The Verus and flux cases define the target band: localization-hard, verification-bottlenecked bugs where the symptom sits far from the cause and the suite cannot see the difference between fixes. Each loop now carries its deductive rung before it runs: testing X, predict Y, refuted by Z (`METHODOLOGY-preregistration.md`, `CANDIDATES-localization-hard.md`). Three to five audited existence cases, or the committed null, is the next paper.
- Coverage as the design lever, the oracle from approved history. The Verus dissection (§7.4, §7.5) names two handles the next experiments turn directly. Gate coverage sets the generalization frontier, so widening the enumeration to the genuine-divergence-preserve shape is a falsifiable test of whether the model then carves out the XOR's hard arm. And the oracle a model cannot induce is mined from human-approved history, the merged fix, the regression suite, the resolved-issue label, so calibrating `abductor` differentially against base and the approved fix is the concrete way to close a gate's blind spot without authoring a fresh oracle by hand. Prospectively, on a bug with no merged fix yet, the hard arm has no golden until a human spends the judgment.
- Conjecture: the XOR is the necessary, and necessarily external, operation of inquiry. Read the dissection one level up. Every warrant-producing step contracts the symmetric difference between what the inquirer believes and what is true, locating one case the current hypothesis mishandles and removing it. Generation cannot find that case, because adding hypotheses has no floor that says this one is missing; only a disagreement evaluated against a reference outside the inquirer's belief can, which is why the externalized oracle was load-bearing and not merely convenient. If that is right, the oracle's source is not a deployment detail but the operation inquiry is made of. The same contraction recurs across the companion posts (refutation in What Cannot Be False Cannot Be True, stranger-replay in Verifiable Knowledge, the fold in Compress and Unfold), which is suggestive rather than evidential and is kept here as a conjecture, not leaned on as a citation. It dies if a recorded trace reaches the same held-out general predicate from self-authored labels only, with no external label, replay, human-approved reference, or environmental execution entering before the discriminating step; lucky guesses (no warrant) and already-solved cases (the difference already empty) do not count.
- Conjecture: in-context abduction degrades with diff size. Hold the harness fixed and the claim is about the operands: a model's ability to compute the XOR in its own context should fall as the diff and the case-space grow, the failure mode Don't Lang What You Can Math already names, a large structured operation attempted in prose rather than executed. Two regimes, and only one is a conjecture. The endpoint is by construction and needs no experiment: once the XOR's operands exceed the context window, in-context computation is impossible, so externalization is the only option at scale rather than the better one, and a codebase outgrows any finite window eventually. The interior, within the window but large, is the falsifiable part, where in-context performance should degrade as the operands grow, and degrade faster than linearly. Computing the XOR is a synthesis task: it relates cases to one another to find the one the hypothesis mishandles, and the pairs to weigh grow with the square of the case-space, so the load rises quadratically where a fact-lookup would rise linearly (Context Synthesis is Quadratic). The underlying fall-off as input grows is the context-rot phenomenon (Chroma 2025); we have no direct measurement of XOR accuracy against diff size, and flag that rather than borrow a result that grades something else. It dies if a within-harness sweep over diff and case-space size shows in-context XOR accuracy flat or rising with size. The Verus receipt is a single point on that axis.
- The hygraph as a type. Specify the abstract data structure independently of this harness: operations (perturb-and-classify, generate-edge-from-kill, prune, replay), the soundness invariant (every node reconstructible from its recorded trial), and a serialization any agent can emit. The goal is an interchange format for auditable reasoning, so that "show your work" becomes a machine-checkable demand rather than a rhetorical one.
- Grade the trace, not just the patch. Bench design follows from the audit: report determinacy-aware denominators; build instruments whose oracle is expensive or absent and whose causes are hidden,

because that is where method separates from reach; and score submissions on replayability, so a suite-green over-narrow fix (§7) stops being indistinguishable from a root-cause one.
- Receipts-first contribution as the deployment lane. The flux submission is the template: fix, discriminating receipt, soundness twin, full suite, replayable graph, residual flagged to the experts. Agent PRs are drowning in justified slop suspicion; the trace is the antidote, because it converts "trust my patch" into "audit my ledger." Scaling that lane (and measuring maintainer response to trace-backed PRs against the 50.6% baseline) tests attestation-displacing-trust ecologically. The first maintainer merge of a trace-backed fix on a maintainer-stuck issue is the lane's golden ticket: an adversarial expert accepting the work on its ledger, with the doer invisible to the verdict.
- Cross-instance smem accumulation. Let the graph grow across instances within a repo, then across repos within a domain; the current work tests the smem only at per-instance scope.
- Concurrent diagnosis over a shared graph. The monotone graph already permits the lock-free fan-out named in §9: a conflict-free blackboard for the inquiry, layered over git's blackboard for the code. Running it, and measuring the wall-clock win, is the to-do.
- Held-out submission. Pro's private split removes the visible oracle, which after §8.5 makes it the honest test of the gate-blind harness, well beyond a formality. We expect a substantially lower number there and regard predicting it in print as part of the discipline.

Beyond SWE-bench, the harness is one chapter of a broader program: a compiler from prose to executable agent behavior, of which the methodeutic skills are the procedural memory, the hygraph the typed semantic memory, and the deterministic gate the runtime check. The program is developed at length in the methodeutics textbook; compiler is used descriptively, in the LLVM (Lattner & Adve 2004) and DSPy (Khattab et al. 2023) lineage of typed pipelines from specification to reproducible behavior.

13 Conclusion

The result is narrow. In the domain where every step is checkable, an agent's reasoning can be made accountable rather than merely trusted. The hypothesis graph is a replayable record that submits each generated step to a world-facing trial and retains only what survives, so the path from question to fix is a sequence of falsifiable commitments a third party can rerun rather than a verdict it must accept. This reduces the cost of relying on an agent from reproducing its work to rerunning its record. The guarantee holds only where the work is perturbable, where a trial can be run and read; extending that region is left open, and the method claims nothing beyond it.

14 Availability and reproducibility

- Repositories. github.com/kimjune01/swebench-pro (the bench run; frozen tags `prereg-pro-v1`, `prereg-pro-v1-cheap`), github.com/kimjune01/swebench-verified (prior-generation baseline, Zenodo-DOI'd), github.com/kimjune01/swebench-pro-audit (the determinacy audit; every claim one row in `CLAIMS.md`, all 728 verdicts in `COVERAGE.md`, mechanical spine re-derivable by grep), github.com/kimjune01/determinacy (the audit as a portable tool for any SWE-bench-shaped bench; SWE-rebench run included), github.com/kimjune01/hygraph-mechanism (the mechanism experiment; the Verus #2219 lift with its clean, forced-fresh dataset and the 43,586-line climb trace, and the flux trail frozen at `flux-1613-trail-v1`; the README's artifact index maps every claim here to its committed path), github.com/kimjune01/abductor (the externalized kill condition as a standalone, domain-general instrument: enumerate, calibrate against a known-good baseline, gate, with the `/debug` skill that drives the loop).
- Provenance artifacts. Per-instance trajectories, hypothesis graphs, captured diffs, gate traces, and cost ledger under `runs/scored/artifacts/`; preregistrations at the freeze SHAs; the OSS ledger (`pr-receipts.jsonl`) with the GraphQL query that recomputes every number it asserts.
- OSS deployment trace. ~385 hypothesis graphs at `kimjune01/sweep/repo-hypotheses/`, one per investigated issue; PR-level outcomes pinned at `kimjune01/kimjune01@paper-2026-05-28`.
- Replication. Boxes, budget, the per-instance cost ledger (`COST_BASIS.md`), and the step-by-step rerun live in the run repo and the field guide How Not to Run SWE-bench Pro.
- Companion writing. The instrument story and field guide: How Not to Run SWE-bench Pro. The error corrected here, from the inside: Precisely Wrong. The epistemology the discussion rests on: Verifiable Knowledge and its frame What Cannot Be False Cannot Be True; its buildable-truth core in brief: Truth Is Buildable. Dated provenance posts establish parallel rather than derivative development: Theory is load-bearing (2026-03-17), The proof manual (2026-04-05), and Type the

question (2026-04-08) predate ADI (2026-04-17); Evidence has a trajectory (2026-04-27) and The Hypothesis Graph (2026-04-28) predate CMM (2026-05-26).
- PDF. Arxiv-shape build at /assets/the-hypothesis-graph-semantic-memory-methodeutics.pdf, rebuilt from this markdown source by md2arxiv; the source is canonical.
- DOI. Prior-generation verified artifact: Zenodo-DOI'd. Mechanism experiment (hygraph-mechanism): 10.5281/zenodo.20691973. Pro bundle (swebench-pro): 10.5281/zenodo.20691977. This paper: DOI pending.
- License. Skills released under CC-BY-SA-NS (june.kim/cc-by-sa-ns); repo-level terms in each `LICENSE.md`. The harness an outsider clones is the same harness that produced the published numbers.

Reproducibility invitation. Nullius in verba. Every number here is recomputable from committed artifacts: the bench verdicts by re-running the official grader on captured diffs, the audit's mechanical spine by grep, the flux divergence by replaying the receipt programs against both committed patches. Doubts should be filed as issues against the relevant repository; confirmed corrections fold into the next versioned artifact, as the retraction noted in §8.6 and the reversal this version reports already have.

LLM collaboration disclosure

LLMs enter this work in three roles. Subject of study: the harness under evaluation uses frontier LLMs as generator and challenger, with versions, billing mode, and provenance disclosed in §8.1 and the artifacts. Instrument: model pairs adversarially verify the audit's two-expert tier (one constructs, an independent family refutes) and blind-judge the mechanism pilots, always with the mechanical layer (grep, grader, replay) holding the verdict. Writing aid: the prose was drafted and revised with Anthropic's Claude (Opus 4.8 and Fable 5) from human-authored outlines and session notes; the claims, methodology, numbers, and argument structure are the author's. No LLM decided what to publish.

Acknowledgments

We thank John Laird for endorsing this submission, and the flux maintainers for engaging with a stranger's receipts on their hardest open issue.

15 Novelty and comparative search protocol

- Why this section exists. Several claims here take the form "we found no prior X." Such claims are only as honest as the search they rest on. This section publishes the queries so readers can re-run the search and either confirm the gap or find what we missed.
- Sources searched. Google Scholar; arXiv (cs.LG, cs.AI, cs.CL, cs.SE); ACL Anthology; OpenReview; GitHub code and repository search; public SWE-bench leaderboard and submission archives.
- Queries by claim.
  - Peircean SE-agent loop. "Peirce" "LLM agent" abduction deduction induction software engineering; "abduction deduction induction" "LLM agent" "software engineering"; "Peirce" "SWE-bench" agent.
  - Hypothesis-graph agent memory. "hypothesis graph" "LLM agent" memory; "belief graph" "LLM agent" memory hypothesis; "knowledge graph" "LLM agent" "hypothesis" "memory".
  - Blind multi-model hypothesis-stage filtering. "multi-agent" "code" "hypothesis" "SWE-bench"; "blind" "multi-agent" "code review" LLM.
  - Trajectory-shape termination gates. "LLM agent" termination criteria trajectory; "finite state" "LLM agent" "termination".
  - Benchmark determinacy / construct-validity audits. "SWE-bench Pro" "construct validity"; "underdetermined" benchmark "problem statement" test; "specification" "ambiguity" "SWE-bench" audit.
  - Full per-instance provenance on SWE-bench Pro. SWE-bench Pro leaderboard submissions trajectories cost ledger; site:github.com "swe-bench-pro" "trajs".
  - Sub-$1k Pro replication and per-instance cost ledger. "SWE-bench Pro" "cost per instance"; "SWE-rebench" cost per problem.
- Caveats. The search is best-effort and bounded by visible-web indexing; private industry work and non-indexed venues are not covered. Discoveries of overlapping prior work should be reported as issues for citation update.

15.1 Comparative search supporting the artifact claim

The artifact claim (§8.2), no method documented publishes per-instance receipts at this depth on SWE-bench Pro, requires a comparative search. The claim concerns receipts, with rate set aside. The bar: published per-instance trajectories, captured diffs, gate or evaluator traces, cost ledger, and reproducible run conditions.

Candidate audit (against the receipt bar). Each top public submission or comparable report is checked for: published per-instance trajectories (T), captured diffs (D), evaluator/gate traces (G), per-instance cost ledger (C), reproducible frozen artifact (R), and resolve rate at or above ours on the same bench. Receipt-bar columns are present (Y), partial (~), or absent (,).

Submission / report	Bench	T	D	G	C	R	Rate >= ours	Notes
Official `swebench/experiments` repo (multiple top entries)	Verified	Y	Y	,	,	~	Various	Minimum publication norm: trajs/logs/patch.diff/report. No gate traces, no cost ledger.
Top vendor leaderboard entries (Claude Code, OpenHands, SWE-agent, AutoCodeRover)	Verified	~	~	,	,	,	Reported below 97%	Submissions report numbers; reproducible bundles and cost ledgers rarely published.
SWE-bench Pro official page (Scale)	Pro	~	~	,	,	,	N/A (curator)	Uncapped cost (250-turn limit). No per-instance cost ledger.
Nilenso Pro trajectory analysis	Pro	~	,	,	~	,	N/A (third-party)	Cost/token/time analysis across four frontier models. Not a submission.
SWE-rebench public reports	rebench	~	~	,	Y	~	Below ours	Strong cost transparency (Cursor Composer 2.5 at $0.23/problem).

Submission / report	Bench	T	D	G	C	R	Rate >= ours	Notes
This work: Verified	Verified	Y	Y	Y	Y	Y	426 / 438 eligible (97.3%)	Companion repo `swebench-verified`; Zenodo DOI; gate traces and cost ledger committed.
This work: Pro	Pro	Y	Y	Y	Y	Y	694/728 = 95.3%; open-weight pair 678/728 = 93.1%	Same frozen harness, whole eligible set, 0 incomplete; two model pairs under one bundle. Public-split, gate-oracle regime: an artifact claim, not a leaderboard claim (§8.3).

Reading. No row above the two rows here combines all five receipt-bar columns (T/D/G/C/R) on the same bench. The claim is about receipt depth alone, leaving resolve rate out of it, and survives as long as the table reads this way; a citation showing a fuller combined receipt is the cleanest refutation.

16 Extended intellectual lineage

Foundational sources grounding §3, §2, and §10, collected here so Related Work stays focused on contemporary systems.

16.1 Peircean inquiry and the philosophy of science

- Peirce 1878 (Illustrations of the Logic of Science): the original three-mode taxonomy.
- Peirce 1903 (Pragmatism as the Logic of Abduction): abduction as the only mode that introduces new content.
- Bacon 1620 (Novum Organum); Popper 1934 (The Logic of Scientific Discovery): falsification as the inductive-side constraint.
- Ramsey 1926 (Truth and Probability): operational credence as betting odds; the hygraph's node-level semantics descends from this work.
- James 1907; Dewey 1929: the pragmatist commitment that truth is inseparable from action.
- Meehl 1967; Feynman 1974 ("Cargo Cult Science"): the difference between rigor-shaped activity and actual rigor, the standard §8.4 holds itself to.
- Kimball 1957; Tukey: the Type III error, the exact answer to the wrong question; the failure mode the determinacy audit (§8.7) is built to prevent, examined in the companion post Precisely Wrong.

16.2 The hygraph's structural ancestors

- de Kleer 1986 (assumption-based truth-maintenance systems): persistent dependency structures over beliefs with mechanical retraction; the TMS keeps consistency where the hygraph keeps trials, and a TMS justification is not replayable by a stranger.

- Dung 1995 (abstract argumentation frameworks): attack relations between claims as first-class structure; the hygraph's kill edges are attack edges bound to executions rather than arguments.
- Wald 1947 (sequential testing) and Vovk & Wang 2021 (e-values): the sequential-evidence framing that shaped `inquire`'s diagnostic stance. No accumulator is deployed: the code under test is deterministic, so the gate routes on the binary grader verdict (§5.4).
- Pearl 1988; 2000/2009: DAGs as the substrate for structured belief and causal-structure inference; this work borrows the typed-node/typed-edge form and leaves the probabilistic semantics behind.
- Zeller & Hildebrandt 2002 (delta debugging): the canonical demonstration that mechanical perturbation of code is a productive inference primitive; optimization-shape where the hygraph is methodology-shape.
- Reiter 1987 (A theory of diagnosis from first principles) and de Kleer & Williams 1987 (the General Diagnostic Engine): model-based diagnosis as conflict sets, minimal hitting-set diagnoses, and entropy-minimizing choice of the next measurement. The hygraph runs this abductive loop but over an open, LLM-generated hypothesis space rather than a fixed component model, and its "where to perturb next" is the GDE measurement-selection problem handed to the reasoner. Reiter's hitting sets are the theory of how multiple kills jointly localize a cause.
- Clarke et al. 2000 (counterexample-guided abstraction refinement, CEGAR), Solar-Lezama et al. 2006 (counterexample-guided inductive synthesis, CEGIS), and Cousot & Cousot 1977 (abstract interpretation): the closest formal kin to the hygraph's loop, in which a counterexample is a kill that names the next refinement. The hygraph generalizes that loop from a closed abstraction domain or synthesis grammar to an open hypothesis space, which forfeits the completeness and sound-by-construction refinement that closure buys (the open-world relevance boundary is undecidable in general, reducing to Rice's theorem) and recovers soundness only per node, through replay. Two pieces of their machinery are unported and natural to inherit: the real-versus-spurious counterexample check (a principled test of whether a kill is genuine or an artifact) and widening (a principled stop that over-approximates with stated precision loss, the formal form of declaring an inquiry's trajectory oscillatory and committing to the general shape).

16.3 Bi-abductive and compositional inference

- Calcagno et al. 2009: compositional shape analysis via bi-abduction; Facebook Infer as the industrial-scale instance of typed-mode inference on real code.
- O'Hearn 2019: separation logic and incorrectness logic.
- Zilberstein, Saliling & Silva 2024 (arXiv:2305.04842): Outcome Separation Logic; tri-abduction for branch composition.
- Bylander et al. 1991: abductive computational complexity.